The Why

The Why of North Star

To make human capability visible and navigable.

Right now the world hides talent, knowledge, and opportunity inside:

- resumes
- job titles
- institutions
- social networks
- keyword searches

All of those are **flat lists** pretending to represent a **multidimensional reality**.

North Star system is the opposite.

It reveals that reality.

Instead of:

People → listed
Skills → tagged
Opportunities → searched

It shows:

People → connected
Skills → evolving
Communities → emerging
Opportunity → discoverable

The Problem It Solves

Humans currently navigate capability like this:

LinkedIn
Google
Resume
Job description

Those are **index systems**.

But capability is **networked**.

The true structure looks like:

person
↳ skills
↳ collaborators
↳ experience
↳ interests
↳ communities
↳ reputation

Which forms a **living graph**.

North Star simply **reveals the map that already exists**.

The Real Product

The visualization is not the product.

The **map is the product**.

The interface is just the telescope.

The Transformational Insight

When people can **see the network**, three powerful things happen:

1. Discovery replaces search

Instead of:

> “Find a data scientist”

You see:

cluster → AI research
bridge node → bioinformatics
emerging cluster → climate modeling

You discover capability you didn’t know to search for.

---

2. Opportunity becomes visible

Hidden bridges appear:

community A
↕
connector person
↕
community B

Those bridges create:

- new companies
- research
- partnerships
- movements

---

3. The invisible becomes visible

Right now:

quiet experts
unrecognized contributors
cross-disciplinary thinkers

are buried.

A graph **elevates them automatically**.

Because the network itself reveals importance.

Why the Universe Metaphor Matters

The concept of **zooming out to a universe** is not just aesthetic.

It expresses truth.

Human capability really behaves like a cosmos.

You have:

stars → individuals
constellations → teams
galaxies → industries
cosmic filaments → knowledge flows

When someone zooms out, they literally see:

the structure of human intelligence.

The Emotional Goal

When someone first uses this system they should think:

> “I didn’t know the world looked like this.”

Not:

> “Cool graph.”

The Behavioral Outcome

The system should change how people make decisions.

Instead of asking:

Who should I hire?
Where should I work?
Who should I collaborate with?
What should I learn next?

They ask:

Where am I in the network?
Where are the interesting clusters?
Where are the bridges?
Where is the frontier?

The Product North Star

The core metric is not:

- nodes
- users
- queries

The metric is:

Meaningful connections discovered.

Or more precisely:

unexpected connections that lead to action

What This Means for the Engine

The architecture should optimize for:

1. Pattern recognition

Users should instantly see:

- clusters
- bridges
- hubs
- emerging zones

---

2. Progressive revelation

Zoom levels should move through:

person
↓
team
↓
community
↓
industry
↓
human capability universe

---

3. Exploration, not search

Search exists.

But exploration is the primary mode.

The North Star Statement

If you had to express it in one sentence:

North Star reveals the hidden structure of human capability so people can discover opportunity that traditional systems hide.

A Test for Every Feature

Before building anything, ask:

Does this help someone see the network more clearly?

If the answer is no, don’t build it.

The Ultimate Vision

If this works at scale, the system becomes:

a telescope for human potential

A place where anyone can zoom out and see:

- where knowledge lives
- where innovation is forming
- where opportunity is emerging

In other words:

a living map of civilization’s intelligence.

Core Idea

---

1. The Core Idea of North Star

North Star is **not a social network**.

It's a **capability graph of builders and operators**, grounded in **demonstrated work**.

Instead of:

- resumes
- bios
- titles

North Star organizes people by:

- **projects**
- **decisions**
- **constraints**
- **failures**
- **metrics**
- **experiments**

It answers the question:

> "Who has actually built something like this before?"

---

2. The Profile (What Yours Looks Like)

Your profile is essentially a **personal operating system**.

Minimal structure:

Profile Layers

Layer 1 — Identity
Who you are and how you think.

Prentiss
Frontier Operator

Focus:
AI-native operating systems
Coordination layers
Decision compression systems
Knowledge acquisition platforms

Layer 2 — Project Ledger

Projects become **nodes**.

GetIT
Fast Food
Anansi

Each project expands to:

- system design
- constraints
- decisions
- failure logs
- metrics
- next proof

These are **high signal artifacts**.

Layer 3 — Pattern Detection

The system extracts patterns across projects.

Example:

Patterns detected:

Infrastructure bias
Long-horizon architecture
Local-first systems thinking
Density-first marketplaces
Agentic modular design

This becomes **machine-readable capability data.**

Layer 4 — Ask Interface

Users can interrogate the profile:

Ask:
“What would Prentiss do in the first 30 days of a struggling marketplace?”

The system answers using the ledger.

Layer 5 — Fit Check

Paste problem → get structured evaluation.

Fit verdict
Evidence mapping
Risks
Questions
First 30 days plan

This replaces resumes.

---

3. What the Platform Looks Like at Scale

Now imagine **thousands of profiles built the same way**.

Instead of LinkedIn’s messy graph, you get **interconnected matrices**.

Think of it like this:

People
Projects
Decisions
Constraints

Metrics

Each becomes a searchable dimension.

Capability Graph Example

A founder searches:

“Operators who built density-first marketplaces”

The system returns profiles where:

- a project contains that decision pattern
- metrics support it
- failures document real experience

Now you’re discovering **real operators**, not self-descriptions.

---

4. The Living Matrix

Here’s where it gets powerful.

North Star connects **people through their projects and decisions**, not through social links.

Example network:

```
Prentiss
  |
  | built
  |
GetIT
  |
  | shares pattern
  |
Density-first marketplaces
  |
  | also used by
  |
```

Other founders

Suddenly you can explore:

- everyone who solved similar problems
- everyone who failed in similar ways
- everyone optimizing for similar constraints

That's the **matrix**.

---

5. How People Interact With North Star

Interaction happens through **four primary modes**.

---

1. Interrogate a Person

Hiring leaders:

Ask:
"What's Prentiss's approach to agent governance?"

They see:

- answer
- linked project decisions
- failure logs

This builds **deep trust quickly**.

---

2. Interrogate a Problem

A founder asks:

"How do people solve cold-start density?"

North Star returns:

- decision patterns
- relevant projects
- operators who solved it

---

3. Discover People

Search becomes **semantic capability discovery**.

Example queries:

local-first system architects
operators who shipped AI agents
marketplace builders with density strategies
edtech builders with mastery metrics

LinkedIn cannot do this.

---

4. Collaborate

Founders discover each other through **shared patterns**.

Example:

Operator A:
density-first marketplaces

Operator B:
trust-layer marketplaces

Operator C:
local-first AI infrastructure

North Star detects compatibility.

---

6. ICPs (Initial Target Users)

Do not start broad.

Your ICP must be **people who benefit from demonstrated work.**

ICP 1 — Technical Founders

Why they love it:

- hire better
- find collaborators
- surface real operators

ICP 2 — Product / Engineering Leaders

Use it for:

- hiring
- capability discovery
- learning from real build logs

ICP 3 — Venture / Operator Investors

They want:

- pattern recognition
- operator discovery
- project insight

ICP 4 — Frontier Builders

People building:

- AI systems
- robotics
- new interfaces
- infrastructure

These people **hate LinkedIn**.

North Star is for them.

---

7. Initial Use Cases

Hiring

Paste role brief → find operators with proven experience.

Founder Discovery

Find collaborators who solved adjacent problems.

Operator Reputation

Your credibility becomes:

decisions made
problems solved
metrics achieved

Not job titles.

Learning

Students and builders learn by exploring **real project logs**.

---

8. Why This Can Become Massive

Because **LinkedIn organizes people by identity.**

North Star organizes people by **capability**.

The difference:

LinkedIn:

Person → title → company

North Star:

Person → projects → decisions → patterns

That’s fundamentally more useful.

---

9. The Product Loop

The growth engine is simple.

Step 1

Builders create high-signal profiles.

Step 2

Hiring leaders and founders interrogate them.

Step 3

Profiles become reference material.

Step 4

More builders join to gain credibility.

Step 5

The capability graph grows.

---

10. The One Rule That Makes It Work

Profiles must be **structured and artifact-backed.**

If profiles become vague:

North Star becomes LinkedIn.

If profiles stay structured:

North Star becomes a **capability OS.**

Final Thought

Your profile isn’t just a page.

It’s the **template for the entire system**.

If someone can create a profile like yours in **10 minutes from uploaded files**, North Star becomes incredibly powerful.

Core Interfaces

Good. We're going to design **three core interfaces** that make North Star feel radically powerful **without forcing people to learn a new paradigm**.

The key principle:

Familiar entry → immersive exploration → precise interrogation

The three screens:

1. **Constellation Canvas** (the capability map)
2. **Node Expansion Panel** (the operator surface)
3. **Ask the Graph** (problem-first exploration)

Think of it like:

Apple Maps + Figma canvas + ChatGPT intelligence.

---

1. The Constellation Canvas

(The capability matrix)

This is the **primary environment** of North Star.

Users aren't scrolling.

They are **exploring a space**.

Layout

```
-----------------------------------------------------------------
| North Star                          Explore                   |
|---------------------------------------------------------------|
|                                                               |
|                                                               |
|              *                                                |
|           *     *      *                                      |
|        *    *      *                                          |
|                                                               |
|            *          *                                       |
|                                                               |
|                                                               |
|---------------------------------------------------------------|
```

| Mode: People | Projects | Decisions | Patterns |

Dark background.

Nodes glow softly.

Clusters slowly breathe.

Nothing frantic.

Node Types

Each node has a **visual identity**.

Person

Circle.

●

Color: soft blue.

Hover preview:

Prentiss
Frontier Operator

Projects:
GetIT
Fast Food
Anansi

Project

Rounded square.

■

Color: warm white.

Hover preview:

GETIT
Coordination Layer for Services

Metrics:
Booking density
Repeat bookings

Decision

Diamond.

◆

Color: violet.

Example:

Density-first marketplaces
Local-first AI
Agent modularization

Pattern

Large faint node.

⬡

Examples:

Infrastructure builders
Marketplace operators
Agent system architects

These appear only when zooming out.

---

2. Interaction Model

The canvas must feel **fluid and intuitive**.

Hover

When hovering:

- node brightens
- immediate context appears

GETIT
Built by: Prentiss
Category: Marketplace Infrastructure

Connections subtly illuminate.

Click

Clicking a node **expands its orbit**.

Example:

```
      ◆ Density-first marketplace
            |
            |
  ● Prentiss ----- ■ GetIT
            |
            |
       ◆ Coordination layer
```

Users instantly see:

- related decisions
- connected projects
- nearby operators

Zoom

Zoom changes **information density**.

Far zoom

You see **patterns**.

Marketplace builders cluster
Agent systems cluster
Education systems cluster

Mid zoom

You see **projects**.

Close zoom

You see **people and decisions**.

Just like **maps**.

---

3. Node Expansion Panel

(The operator surface)

When a user clicks a node, a **panel slides in from the right**.

The constellation remains visible.

This preserves context.

Person Panel

Example: **Prentiss**

Prentiss

Frontier Operator

Patterns detected
Infrastructure-first
Long-horizon architecture
Local-first systems

Projects
• GetIT
• Fast Food
• Anansi

Decisions
• Density-first marketplaces
• Coordination layer vs listings
• Local-first AI

Metrics used
• Booking density
• Decision acceptance rate
• Mastery lift

Ask about this operator
[What would they do in a stalled marketplace?]
[How do they approach AI agents?]

Run Fit Check

Everything here comes from **Build Ledger data**.

No fluff.

Project Panel

Example: **GetIT**

GETIT
Coordination Layer for Services

System design
Video trust → booking → payment
Context layer + modular agents

Key decisions
Density-first marketplaces
Coordination vs listings
Ambient architecture

Metrics
Booking density
Repeat booking rate
Time-to-intent fulfillment

Next proof
Houston micro-area density test

Users can then expand deeper:

Open Build Ledger

---

4. Ask the Graph Interface

This is where North Star becomes **intelligent exploration**.

Instead of asking about a person, users ask **about a problem**.

Interface

Top bar:

Ask North Star

How do builders solve marketplace cold start?

When submitted:

The **graph reacts**.

Nodes glow.

Clusters shift.

Connections illuminate.

Example Visualization

```
◆ Density-first marketplaces
    |
    |
■ GetIT ------- ● Prentiss
    |
    |
■ Other marketplace project
```

Right panel explains:

Patterns found

1. Density-first micro-area strategy
2. Trust loops before paid growth
3. Liquidity before expansion

And links to relevant ledgers.

---

5. Exploration Modes

Bottom bar toggle:

People | Projects | Decisions | Patterns

Switching modes reorganizes the constellation. It and the top bar become translucent and glassmorphic as you explore deeper into the clusters.

People Mode

Users see operators.

Connections represent:

- shared projects
- shared decisions
- collaboration

Project Mode

Users see projects.

Connections represent:

- shared architecture
- shared market types
- shared metrics

Decision Mode

Users see decisions.

Example:

Local-first AI
Agent modularization
Density-first marketplaces

Users immediately see:

- who chose them
- which projects used them

Pattern Mode

Users see emergent thinking clusters.

Example:

Infrastructure builders
AI agent architects
Marketplaces
Edtech systems

This becomes incredibly powerful.

---

6. The Emotional Design Layer

The interface should feel:

calm
intelligent
expansive

Not:

noisy
busy
startup-y

Visual references

Good inspiration:

Apple Maps
Figma canvas
Notion AI
Linear UI
Obsidian graph

---

7. Motion Design

Motion should communicate meaning.

Examples:

Node Discovery

Nodes **fade in like stars appearing**.

Connection Reveal

Edges **pulse once** when highlighted.

Graph Reorganization

Clusters **drift slowly** when changing modes.

Not snap transitions.

---

8. The First-Time Experience

When a new user arrives:

They see the constellation.

Then a subtle onboarding:

Explore capability.

Search a problem.
Click a node.
Zoom to discover patterns.

Three steps.

No tutorial.

---

9. Why This Interface Works

It satisfies three core human instincts:

Curiosity

People want to explore.

Pattern recognition

Humans detect relationships visually.

Agency

Users feel like **discoverers**, not consumers.

---

10. The Long-Term Potential

Once thousands of profiles exist, the constellation becomes:

A living map of human capability.

You can explore:

- markets
- technologies
- systems
- operators

All through **relationships**.

The Most Important Design Constraint

Never sacrifice **clarity for spectacle**.

If users ever say:

> “This looks cool but I don't know what I'm looking at”

the product fails.

The map must always answer:

Who built this?
What did they decide?
What worked?

Core Interface v1

These three screens must do one thing well: **make human capability understandable at a glance, explorable in depth, and queryable with intent**.

If these three work, the product works. Everything else is infrastructure.

Below is the **UI blueprint for the MFP**, written so design + engineering can implement without ambiguity.

North Star — Core Interface v1

Three primary surfaces:

1. **Constellation Canvas** (exploration)
2. **Node Expansion Panel** (understanding)
3. **Ask-the-Graph Interface** (discovery)

Everything else should stay minimal.

---

1. The Constellation Canvas

Purpose

This is the **main environment**.

It must instantly communicate:

- this person's work
- clusters of capability
- relationships between ideas, decisions, and outcomes

The canvas should feel like **a calm universe**, not a dashboard.

Layout

```
| North Star Logo      Search/Ask       Profile     |
-----------------------------------------------------
|                                         |
|                                         |
|          (Constellation Graph Environment)        |
|                                         |
|                                         |
|                                         |
|                                         |
|                                         |
-----------------------------------------------------
|  Legend | Zoom | Reset View | Share              |
-----------------------------------------------------
```

No sidebars.

No clutter.

The graph owns the screen.

Visual Language

Nodes float in a dark neutral field.

Colors represent **node types**.

Node	Color	Shape
Person	soft gold	star
Project	electric blue	sphere
Decision	violet	diamond
Skill	teal	small dot
Outcome	amber	square

Edges are **subtle glowing trails**.

Clusters naturally form around projects.

Default View (Initial Load)

Camera centers on the **Person node**.

Surrounding it:

```
Person
  |
Project clusters
```

Example:

```
        GetIT
          *
         / \
        /   \
     Decisions Skills
          |
       Outcomes
```

Each project becomes its own **mini constellation**.

Interaction

Hover Node

Effect:

- node glow increases
- connected edges highlight
- connected nodes brighten slightly

Purpose:

Show relationship structure.

Click Node

Triggers:

Node Expansion Panel.

Camera subtly centers node.

Connected cluster becomes emphasized.

Zoom Out

Projects compress into clusters.

Eventually only:

- Person node
- project clusters

remain visible.

At far zoom, **ghost constellations appear** (future network hint).

Drag Canvas

User can explore freely.

Movement should feel:

- smooth
- frictionless
- slightly inertial

Like moving through space.

Micro-Reward Mechanics

When nodes are added or improved:

- glow pulse
- cluster density increases
- edges animate briefly

This reinforces:

your constellation is evolving.

---

2. The Node Expansion Panel

Purpose

The graph shows structure.

The panel shows **meaning**.

This is where users understand **what the node actually represents**.

Trigger

Clicking any node opens the panel.

Panel slides from the right.

Graph remains visible behind.

Layout

```
----------------------------------------
| Node Title                           |
| Node Type Badge                      |
----------------------------------------
| Description                          |
|                                      |
----------------------------------------
| Why This Matters                     |
|                                      |
----------------------------------------
| Connected Nodes                      |
|                                      |
----------------------------------------
| Evidence / Artifacts                 |
```

| |

| Gravity Factors |

Example — Project Node

GetIT

Badge: Project

Description

Video-first coordination layer for local services.

Focus: trust-driven discovery, booking, and payments.

Key Decisions

• Video-first discovery
• Density-first expansion strategy
• Ambient agent infrastructure

Outcomes

• Houston pilot cluster
• Vendor onboarding system
• Transaction loop design

Connected Skills

Marketplace design
Trust systems
AI personalization

Gravity Explanation

High gravity due to:

- 3 major decisions
- 2 measurable outcomes
- 4 supporting artifacts

Panel Behavior

Panel content should be **narrative**, not technical.

Visitors should learn:

- what happened
- why it mattered
- what decisions shaped it

Navigation

Users can click connected nodes inside the panel.

When clicked:

1. graph shifts focus
2. panel updates content

This creates **exploration flow**.

---

3. Ask-the-Graph Interface

Purpose

This converts the graph from visualization into **a thinking engine**.

Users ask questions about capability.

The graph answers using structure.

Location

Persistent field at top of screen.

Ask the graph...

Example Queries

For a single node:

What decisions shaped GetIT the most?

What skills show up across this person’s projects?

What patterns appear in this builder’s work?

Response Format

Results appear as **highlighted graph paths**.

Example:

User asks:

> What decisions shaped GetIT?

Graph highlights:

GetIT → Decision nodes → Outcomes

Nodes pulse.

Panel shows explanation.

Multi-Node Queries (Future)

Example:

Find builders similar to this one

Graph would highlight similar constellations.

This hints at future network capability.

Response Panel

Below the question:

```
---------------------------------------
Insight
---------------------------------------
```

Three decisions shaped GetIT:

1. Video-first discovery
2. Density-first market entry
3. Ambient agent architecture

These connect to two measurable outcomes.

The graph simultaneously highlights those nodes.

The Interaction Loop

These three screens form the full experience.

Constellation Canvas
↓
Node Expansion Panel
↓
Ask-the-Graph

Users move between:

- **seeing structure**
- **understanding meaning**
- **asking questions**

The Key UX Goal

Within **30 seconds**, a visitor should think:

> “I understand what this person builds.”

Within **2 minutes**, they should think:

> “This is a fascinating way to represent work.”

Within **5 minutes**, they should want:

> “I want my own constellation.”

If that happens, the growth flywheel begins.

One Critical Design Constraint

The interface must feel:

- calm
- intelligent
- spacious
- non-corporate

Avoid:

- dashboard clutter
- LinkedIn-style feeds
- resume layouts

North Star should feel closer to:

- a planetarium
- a knowledge atlas
- a capability map

The Single Most Important Detail

When someone shares their link, the first reaction must be:

> “Whoa… what is this?”

That moment is what drives growth.

If you'd like, the next thing worth designing is **the Constellation Rendering Engine** — the layout algorithm and visual physics that make the graph look like a *real galaxy* instead of a typical force-directed network diagram.

The First Node

Good. The **first node decision** is the most important product decision in the entire system. If you get this wrong, the universe becomes noise.

We need to decide **what the atom of the system is**.

Everything else — clusters, constellations, galaxies — emerges from that.

The First Node Problem

A graph is only as good as its **atomic unit**.

Most platforms choose badly.

Examples:

LinkedIn
Atomic unit → **person**

GitHub
Atomic unit → **repository**

Twitter/X
Atomic unit → **post**

Google
Atomic unit → **page**

Those choices shape the entire ecosystem.

What Your Graph Is Actually About

Your system is not about:

- people
- jobs
- resumes
- companies

It is about **capability**.

Capability lives **inside people but connects across them**.

So the atom should not be the person.

The Correct Atomic Unit

The atomic node should be:

A Capability Signal

A capability signal is:

A demonstrated ability to do something.

Examples:

- built a machine learning model
- designed a logo system
- ran a marketing campaign
- performed a surgical technique
- taught calculus
- repaired a diesel engine

These are **proof of capability**, not claims.

Why This Matters

If the node is **person**, you get a social network.

If the node is **capability**, you get a **knowledge map**.

And that difference changes everything.

The Graph Structure

Instead of:

person → person

You get:

person → capability → capability → person

Example:

Alice
↳ Python
↳ Data Visualization

Bob
↳ Python
↳ Climate Modeling

The system automatically reveals:

Python cluster

Which becomes a constellation.

What the First Node Looks Like

When someone enters the system they create **one capability node**.

Not a profile.

Example entry:

Capability:
"Designing multi-location restaurant brand systems"

Metadata attached:

evidence
experience level
tools used
projects
collaborators

The **person node becomes secondary**.

It simply owns capabilities.

Graph Model

Nodes:

Capability
Person
Project
Concept
Organization

But the **primary gravity node** is:

Capability

Everything else orbits it.

What Appears in the Graph

Node types

Person node
small star

Capability node
bright star

Cluster
constellation glow

Example visual structure:

```
[Person]
   |
   | possesses
   |
[Capability] ---- related to ---- [Capability]
   |
   | used in
   |
[Project]
```

This creates real knowledge topology.

The Magical Effect

When thousands of capabilities exist, the graph reveals:

Clusters like:

robotics
AI safety
urban agriculture
quantum computing

And bridges like:

machine learning ↔ climate modeling

Those bridges are where innovation happens.

What the First User Actually Does

Step 1
User enters **3–5 capabilities they can actually do.**

Step 2
System finds related capabilities.

Step 3
Connections appear.

Step 4
Clusters begin forming.

The universe grows organically.

How the Graph Stays High Quality

Capability nodes require **evidence signals**.

Possible signals:

project
publication
code repo
portfolio
peer confirmation

The system weights connections based on signal strength.

The Real Flywheel

As capabilities accumulate:

more connections
→ better clusters
→ better discovery

→ more users
→ more capabilities

Which makes the map more accurate.

What the Very First Screen Should Be

Not:

Create profile

Instead:

What can you actually do?

Followed by:

Add 3 capabilities.

Then the first constellation appears.

That moment needs to feel **magical**.

The First Universe Seed

Before public launch you seed the graph with:

~10,000 capability nodes

Across domains like:

engineering
design
medicine

science
craft
education
entrepreneurship

Then users attach themselves to those stars.

The Core Insight

People change jobs.

People change companies.

But **capability persists**.

Your graph maps the thing that actually matters.

The Final North Star Principle

Everything in the system must answer one question:

What can humanity do?

Not:

Who works where

But:

What capabilities exist
and how they connect.

That's what turns your system into a **map of human potential**.

MFP (Build 1st)

Below is the **CTO-grade instruction document** for the **North Star MFP: One Node System**.

This is written as if you are handing it directly to your **CTO / engineering lead**.
The focus is **building the atomic unit perfectly** before scaling to a network.

North Star

Minimal Fundable Product (MFP)

Atomic Unit: One Node System

Version: v0.1
Scope: **Single Capability Constellation (Founder Node)**
Owner: Product / Architecture
Audience: CTO / Engineering Team

---

1. Objective

The goal of this build is **not a network**.

The goal is to create the **perfect atomic unit** of the North Star system:

> A single human capability constellation that is so compelling it proves the model.

This atomic unit must demonstrate:

1. Capability mapping
2. Graph relationships
3. Exploration value
4. A glimpse of a much larger universe

If the **one node system works**, replication becomes trivial.

---

2. Product Definition

North Star v0 consists of:

1 person
multiple projects
decisions
skills
outcomes
connections

Rendered as a **capability constellation**.

The constellation must allow visitors to understand:

What this person builds
How they think
What decisions shaped their work

within **2–3 minutes of exploration**.

---

3. Core Principle

We are building **the atomic unit of the capability graph**.

The system must answer:

> Can one person's work be represented as a living constellation?

If yes:

1 node → 100 nodes → 10M nodes

The architecture must support this trajectory.

---

4. Core System Components

The system has **five functional layers**.

Data Layer
Graph Layer
Rendering Layer
Interaction Layer
Narrative Layer

---

5. Data Model (Atomic Schema)

Keep the schema extremely minimal.

Node Types

Person
Project
Decision
Skill
Outcome
Idea (optional)

Edge Types

Person → Project
Project → Decision
Project → Skill
Decision → Outcome
Person → Person (future)

Tables

nodes
edges
profiles
projects
skills

decisions
outcomes

Node Schema

id
type
title
description
created_at
metadata

Edge Schema

id
source_node_id
target_node_id
relationship_type
weight
created_at

---

6. Founder Node Data

The first constellation is **Prentiss (Founder)**.

Example node structure:

Person: Prentiss

Connected to:

Projects:
GetIT
Fast Food
Anansi
Black Bolt

Each project connects to:

Decisions
Skills
Outcomes

Example:

GetIT → Decision → Video-first marketplace
GetIT → Skill → Marketplace architecture
GetIT → Outcome → Vendor onboarding

---

7. Graph Engine

The graph engine must:

ingest nodes
calculate layout
render relationships
handle zoom levels

Recommended libraries:

React Three Fiber
Three.js
or
Sigma.js

Layout method:

force-directed layout

Key requirements:

smooth movement
node clustering
edge animation

---

8. Rendering Architecture

Rendering must feel **alive**.

Nodes must visually encode meaning.

Node Types (Visual)

Node	Style
Person	bright core
Project	medium orb
Skill	small orb
Decision	diamond
Outcome	square

Edges

Edges render as:

particle trails

Hover animation:

glow + highlight connected nodes

Cluster Visualization

Project nodes form **mini constellations**.

Example:

Prentiss

|
GetIT cluster
Fast Food cluster
Anansi cluster

---

9. Zoom Levels

Zoom must communicate **conceptual hierarchy**.

Level 0 — Universe Hint

The user sees **one constellation**.

But faint ghost nodes appear beyond.

Text hint:

"More constellations coming."

Purpose: imply the future network.

Level 1 — Profile Constellation

Main working level.

Displays:

Person node
project clusters
connections

Level 2 — Project Cluster

Clicking a project expands:

project node
decisions
skills
outcomes

Level 3 — Node Detail

Overlay panel shows:

node description
connected nodes
context

---

10. Interaction Model

Interactions must feel intuitive.

Mouse Controls

scroll → zoom
drag → move canvas
click → focus node
hover → highlight edges

Focus Mode

Clicking a node:

center camera
fade unrelated nodes
expand cluster

---

11. Onboarding (Founder Only)

This version uses **manual seeding**.

We manually input:

projects
skills
decisions
outcomes

Future version uses AI ingestion.

---

12. Narrative Layer

This system must communicate a story.

Each project cluster must answer:

What problem was solved
What decisions mattered
What outcomes happened

This transforms the graph into a **thinking map**.

---

13. Performance Targets

initial load < 2s
60fps animation
<500 nodes
<2000 edges

This is sufficient for the atomic unit.

---

14. Key Visual Goal

The constellation must feel:

calm
beautiful
intelligent
explorable

User reaction target:

> “This is the most interesting profile I’ve ever seen.”

---

15. Exploration Test

We measure success if users:

explore >3 minutes
click multiple nodes
navigate clusters

This proves **graph curiosity**.

---

16. Hinting at the Universe

The system must subtly show that:

this is one node of many

Techniques:

- ghost nodes in distance
- dim constellations
- tooltip:

"More builders coming soon"

This creates **future scale imagination**.

---

17. MFP Stack

Frontend

Next.js
React
Three.js / React Three Fiber
Tailwind

Backend

Node API
Supabase Postgres

Graph API

GET /profile/{id}/graph
GET /node/{id}

---

18. Build Plan

Week 1

Graph renderer prototype.

Week 2

Node and edge schema.

Week 3

Constellation rendering.

Week 4

Node interaction system.

Week 5

Narrative panel system.

Week 6

Founder constellation population.

---

19. Definition of Done

The atomic node is complete when:

1. Founder constellation renders smoothly
2. Project clusters feel meaningful
3. Users explore multiple nodes
4. The graph suggests a much larger universe

---

20. The Strategic Reason

We must **perfect the atomic unit**.

Because the platform scales as:

1 constellation
100 constellations

10,000 constellations
10M constellations

If the single node is not compelling:

The network will not matter.

---

21. Final Principle

North Star is **not a profile builder**.

It is a **map of human capability**.

The one node system must prove that.

Phased Build Spec

Good instinct.
The product should **wow with one profile** and **hint at the universe**, not try to simulate a universe that doesn't exist yet.

Think of this as:

Phase 0: One Operator Surface (yours)
Phase 1: Query-generated micro-graph
Phase 2: Real capability matrix

And the UI should quietly signal the larger structure:

> Some nodes are real.
> Some nodes are "future constellation points."

Below is the **phased build spec with tickets + acceptance criteria** for the hybrid plan, optimized around **one profile first**.

PHASE 0 — The Operator Surface (Your Profile)

Goal

Create a profile so strong it stands alone as a product.

Duration

~1 week

Core concept

Everything should be navigable from:

Prentiss
↓
Projects
↓
Decisions / Constraints / Metrics / Failures

No graph yet.

Just **clean exploration**.

Ticket 0.1 — Project Ledger Rendering

Description

Render the MDX project files as structured operator pages.

Implementation

Next.js + MDX loader.

Structure:

/projects/getit
/projects/fast-food
/projects/anansi

Each page contains:

- system design
- decisions
- constraints
- failure log
- metrics
- next proof

Acceptance Criteria

Page loads with:

Project title
System design
Decision cards
Failure log
Metrics
Next proof

Decision cards are collapsible.

Anchor links exist for:

#system-design
#decisions
#failure-log
#metrics

Ticket 0.2 — Profile Hub Page

Description

Landing page for the operator.

Implementation

Layout:

Hero
Operator summary
Projects grid
Patterns detected
Ask interface entry

Acceptance Criteria

The page includes:

Prentiss
Frontier Operator

Project cards show:

GetIT
Fast Food
Anansi

Clicking opens project pages.

Ticket 0.3 — Pattern Extraction (Static for v1)

Description

Show patterns detected across projects.

This is a **manually defined list** for now.

Example:

Infrastructure-first thinking
Density-first marketplaces
Local-first systems
Agent modularization

Acceptance Criteria

Patterns appear on the profile page.

Each pattern is clickable.

Clicking shows:

Which projects demonstrate this pattern

Ticket 0.4 — Ask Me Anything (RAG)

Description

Chat interface grounded in:

MDX project pages

Implementation

Pipeline:

MDX
↓
chunk
↓
embedding
↓
vector store

Prompt requires citations.

Acceptance Criteria

User asks:

What is GetIT's density strategy?

Response includes:

Answer
Evidence links
Tradeoffs
Follow-up questions

Links jump to ledger anchors.

PHASE 1 — Micro Constellation (Query-Scoped Graph)

Goal

Introduce the **capability matrix visually**, but only for nodes that exist.

Other nodes appear as **future stars**.

Duration

1–2 weeks

Ticket 1.1 — Canvas Engine

Description

Build the constellation canvas.

Implementation

Use:

React Three Fiber
Three.js

Nodes render in 3D space.

Initial node types:

Person
Project
Decision

Acceptance Criteria

Canvas renders with:

Prentiss node
GetIT
Fast Food
Anansi
Decision nodes

Nodes float gently.

Edges show relationships.

Ticket 1.2 — Node Interaction

Description

Enable interaction.

Behavior

Hover:

node glow
preview card

Click:

right panel opens

Panel content depends on node type.

Acceptance Criteria

Clicking **GetIT** shows:

System design
Key decisions
Metrics
Next proof

Clicking **Prentiss** shows:

Projects
Patterns
Ask interface

Ticket 1.3 — Future Node Placeholders

Description

Nodes that represent **future network participants**.

Example:

Marketplace operators
Local-first AI builders
EdTech systems

Clicking them shows:

North Star is building the capability matrix.
This node will eventually connect operators working on this pattern.

Acceptance Criteria

Placeholder nodes:

- visually distinct (dim glow)
- clickable
- open explanatory panel

Ticket 1.4 — Ask the Graph

Description

Query highlights relevant nodes.

Example:

How do builders solve marketplace cold start?

Graph highlights:

GetIT
Density-first decision
Prentiss node

Acceptance Criteria

Query triggers:

node highlight
connection glow
right panel explanation

PHASE 2 — Edge Persistence (Hybrid Plan Begins)

Goal

Introduce stored relationships.

Duration

~1 week

Ticket 2.1 — Edge Table

Schema

edge
id
from_type
from_id
to_type
to_id
edge_type
weight
meta

Edge types:

built
contains
supports

Acceptance Criteria

Graph loads edges from DB instead of computing them in runtime.

Ticket 2.2 — Claim Node Integration

Description

Claims become nodes.

Claim types:

decision
constraint
metric
failure

Acceptance Criteria

Clicking a decision node shows:

Decision description
Tradeoff
Reversal trigger
Projects using it

PHASE 3 — Early Pattern Layer

Goal

Introduce pattern clustering.

Duration

~1 week

Ticket 3.1 — Pattern Table

Schema:

pattern
id
label
description

Examples:

density-first marketplaces
local-first AI
agent modularization

Ticket 3.2 — Pattern Membership

pattern_membership
pattern_id
node_type
node_id
strength

Acceptance Criteria

Graph can switch to:

Pattern Mode

Nodes cluster around patterns.

PHASE 4 — Early Network Expansion

Goal

Allow the second and third operators to join.

Duration

Later

Ticket 4.1 — Upload → Ledger Generation

Users upload:

docs
slides
repos
notion exports

LLM generates:

project
claims
decisions
metrics
next proof

User confirms.

Ticket 4.2 — Profile Publication

New profiles appear as nodes.

Connections auto-generated via:

claim similarity
pattern membership

Why This Approach Works

You avoid the biggest platform mistake:

building the network before the content.

Instead you build:

1 extraordinary profile
↓
an exploratory interface
↓
a visible universe forming around it

Users feel:

> “I’m early to something.”

One Important UX Detail

Your constellation must always show:

center node = Prentiss

Everything radiates from it.

Future nodes orbit it.

This subtly communicates:

This is the first star.
More are coming.

Spec- Constellation Graph Engine

Constellation Graph Engine — Rendering Architecture Spec

CTO-ready engineering handoff for an immersive, vibrant, zoom-progressive “human capability universe” graph.

BLUF

This system renders a single knowledge node that progressively zooms out into a constellation field and then a galaxy-scale universe. The graph math (nodes/edges) is solvable; the differentiator is a **GPU-first rendering pipeline** with **LOD (level-of-detail) switching**, **additive glow**, **cluster hulls**, **flow fields**, and **cinematic camera motion** that preserves clarity under scale.

---

0. Stack Confirmation

If your stack differs, translate equivalents. I’m assuming:

- **Platform:** Web (desktop-first; mobile compatible later)
- **Renderer:** WebGL2 via **Three.js** (or PixiJS if you prefer pure 2D)
- **Graph tooling:** Graphology + ForceAtlas2 (CPU) for small/medium; GPU layout optional later
- **Shaders:** Custom GLSL for nodes/edges glow + additive blending
- **State/UI:** React + Zustand (or equivalent)
- **Perf target:** 60fps on Apple Silicon; 30fps fallback on low-end

If you’re not WebGL-capable in your current target, stop and fix that first. This concept dies in Canvas2D.

---

1. Visual Description (Objective)

System presents:

- A dark, near-black background

- Glowing colored nodes (stars)
- Curved, glowing edges (connections)
- At close zoom: readable nodes and pulses along edges
- At mid zoom: clusters form meshes/hulls, labels fade out
- At far zoom: nodes aggregate into nebulas/galaxies and long inter-cluster streams

---

2. Engineering Classification (Components)

Rendering Core

- **GraphRenderer (Composite, Controlled, Stateful)**
    - Owns render loop, scene graph, cameras, post-processing, LOD switching

Data Core

- **GraphStore (Atomic, Controlled, Stateful)**
    - Nodes/edges, attributes, indexing, derived cluster membership

Layout Core

- **LayoutEngine (Composite, Uncontrolled during run, Stateful)**
    - Force simulation for local view; cached positions for mid/far

Interaction Core

- **PickingSystem (Composite, Controlled, Stateful)**
    - Hover/click selection, lasso selection (optional), tooltips

Camera/Motion Core

- **CameraController (Composite, Controlled, Stateful)**
    - Smooth zoom/pan/orbit; focus transitions; reduced-motion compliance

Level-of-Detail (LOD) Core

- **LODManager (Composite, Controlled, Stateful)**
    - Switches render modes + geometry density based on zoom + node count

Clustering Core

- **ClusterEngine (Composite, Controlled, Stateful)**
    - Community detection, hull generation, cluster stats, hierarchical grouping

---

3. State Model

Global State

- `zoomLevel` (float, derived from camera distance)
- `lodTier` (enum: `UNIVERSE | CLUSTER | LOCAL`)
- `selectedNodeId` (nullable)
- `hoverNodeId` (nullable)
- `selectedClusterId` (nullable)
- `motionPreference` (enum: `FULL | REDUCED`)
- `themePalette` (enum / token set)
- `filters` (skills, geography, tags, etc.)

Local UI States (per node/cluster)

- `isHighlighted`
- `isFaded`
- `isPinned`
- `confidence/strength` (affects alpha, width, pulse intensity)

Triggers

- Zoom -> LOD transition
- Hover -> highlight neighborhood
- Click -> camera focus + expand detail
- Filter change -> rerun clustering/layout (debounced)

Persistence

- Session: last camera state, filters, selected node
- Backend (optional): saved “views”, pinned nodes, curated clusters

---

4. Layout & Constraint System

Viewport + Safe Area

- Full-screen canvas/WebGL surface

- UI overlays (cards/tooltips) in HTML layer above canvas
- Z-order:
    1. WebGL universe
    2. WebGL highlights
    3. HTML UI overlay
    4. Modal panels (search/filter)

Spacing & Scale Tokens (baseline)

- Node radius (local): `6–10px`
- Node radius (cluster): `18–40px`
- Edge width (local): `1–2px`
- Edge width (streams): `4–10px`
- Glow radius:
    - local nodes: `12–22px`
    - clusters: `40–120px`
    - streams: `30–80px`
- Label size: `10–12px` (fade out by mid zoom)

Breakpoints

- Desktop: full experience
- Tablet: reduce label density + lower particle count
- Mobile: default to CLUSTER tier first, local detail on tap-focus

---

5. Motion Specification

Camera

- **Zoom/Pan:** smooth damping (inertia)
- **Focus transition:** 900ms
- **Easing:** cubic out
- **Interruptible:** Yes
- **Reduced motion:** snap focus (150ms) + disable particle drift

Local pulses (edge data flow)

- **Type:** timing loop
- **Duration:** 1200ms per loop
- **Easing:** linear for dash offset; glow easing for pulse head
- **Interruptible:** Yes

- **Performance cost:** moderate (prefer shader-driven)

Cluster “breathing”

- **Type:** subtle scale + alpha modulation
- **Duration:** 4000–6000ms
- **Amplitude:** scale `1.02–1.06`
- **Interruptible:** Yes
- **Perf cost:** low (uniform updates)

Universe drift (nebula particles)

- **Type:** noise field advection
- **Step:** fixed dt in shader
- **Speed:** extremely slow (feels alive, not animated)
- **Reduced motion:** off

---

6. Implementation Strategy

Simplest Viable Implementation (that still hits the concept)

Three.js + custom shaders:

- Nodes: instanced billboards (point sprites) with radial gradient glow shader
- Edges: line segments in GPU buffer + curve approximation via polyline segments
- Pulses: shader-based dash offset (no per-edge CPU animation)
- LOD: three render modes with different geometry + effects

Avoid heavy graph visualization frameworks that fight your art direction.

Tokenization

Create a token file:

- `colors.*`
- `glow.*`
- `alpha.*`
- `edge.*`
- `motion.*`
- `lodThresholds.*`

Example tokens

- `lod.local.zoomMin = 3.8`
- `lod.cluster.zoomMin = 1.6`
- `lod.universe.zoomMax = 1.6`

Reusability Mapping

- Nodes: same renderer, different “style profile” per LOD
- Edges: same buffer, different shader/material per LOD
- Clusters: separate geometry layer toggled by LOD

---

7. Rendering Architecture (Core)

Scene Graph Layers (ordered)

1. **Background pass**
    - subtle vignette + noise texture
2. **Universe particles pass**
    - far zoom only (instanced points)
3. **Cluster hull pass**
    - mid zoom (alpha shapes / convex hull meshes)
4. **Edges pass**
    - local + mid zoom (curves/lines)
5. **Nodes pass**
    - all zooms (but aggregated at far)
6. **Highlights pass**
    - selection ring, neighborhood glow
7. **Post-processing**
    - bloom + mild chromatic aberration (optional, restrained)

Post-Processing (minimum)

- Bloom: required for “cosmic” look
- Tone mapping: required to avoid blown-out neon
- Additive blending: required for glow layering

---

8. LOD System (The Secret Sauce)

You need **explicit mode switching**, not “just scale the same graph.”

LOD Tier: LOCAL

Goal: readable graph, precise edges, pulses, labels.

Render:

- Individual nodes (N)
- Edges (E) within neighborhood and filtered subsets
- Pulses on important edges
- Labels on selected node + immediate neighbors only

Hide:

- Hulls (unless debug)
- Nebula particles

Rules:

- Cap visible edges: `maxEdgesOnScreen = 15k` (hard cap)
- If E > cap → render only:
    - edges connected to selected/hover node
    - edges above strength threshold

LOD Tier: CLUSTER

Goal: show communities + structure; reduce clutter.

Render:

- Cluster nodes (K)
- Cluster hulls/glows
- Mesh triangles (optional, low alpha)
- Inter-cluster edges (compressed)

Hide:

- individual labels
- most individual nodes

Rules:

- Cluster node size = f(member count)
- Inter-cluster edge width = f(relationship strength)

LOD Tier: UNIVERSE

Goal: galaxies + streams + “human capability cosmos.”

Render:

- Particle nebula per cluster (instanced points)
- Long “streams” between top clusters (spline ribbons)
- No individual edges/nodes

Rules:

- Display only top N clusters by size/centrality (e.g., 200–800)
- Everything else becomes background particle dust

---

9. Data Model (Minimal)

Node

- `id: string`
- `x,y,z: number` (z optional; fake depth can be derived)
- `type: enum` (person, skill, org, concept)
- `clusterId: string|null`
- `colorToken: string`
- `size: number` (importance)
- `tags: string[]`
- `metrics: { confidence, activity, strength }`

Edge

- `id: string`
- `source: nodeId`
- `target: nodeId`
- `weight: number`
- `kind: enum` (similarity, collaboration, prerequisite, etc.)
- `colorMode: enum` (source->target gradient or cluster->cluster)

Cluster

- `id: string`

- `memberCount: number`
- `centroid: vec2/vec3`
- `palette: { primary, secondary }`
- `hull: vec2[]` (mid zoom)
- `galaxySeed: number` (far zoom particle distribution)

---

10. Clustering + Hierarchy

Required

- Community detection (Louvain or Leiden)
- Hierarchical grouping:
    - cluster → meta-cluster → region

Outputs

- `clusterId` per node
- `centroid` per cluster
- `interClusterMatrix` compressed relationships

Hull Generation

- Compute alpha shape or convex hull around cluster member positions
- Smooth hull (Chaikin) before rendering

---

11. Edge Beauty System (How to Make It Beautiful)

The team can “code connections.” Here’s the art system they need.

Curves

- Use a control point offset perpendicular to the source-target vector
- Offset magnitude = f(distance, edgeWeight) with clamp

Glow + Gradient

- Material uses:
    - base line

    - outer bloom mask (via post-processing)
- Color:
    - blend source and target node colors
    - or cluster palette gradient

Pulse

- Use dashed UV along line
- Animate `dashOffset` uniform
- Pulse intensity tied to `edge.weight` or real-time activity

Density Management

Edges should never create “hairball fog” in LOCAL tier.
Hard rules:

- if node not selected and not in neighbor set -> edges fade to alpha 0.05
- if zooming out -> edges crossfade to cluster edges

---

12. Picking / Interaction (WebGL + HTML Overlay)

Picking Approach

- GPU picking via offscreen render target (ID encoded per node)
- Or CPU spatial index (quadtree) for smaller counts

Interaction Rules

- Hover: highlight node + first-degree neighbors
- Click:
    1. set selected
    2. camera focus transition
    3. pin neighbor edges above threshold
- Double click: “drill-in” (force LOCAL tier)

---

13. Performance Audit

Risks

- Excessive draw calls
- CPU layout recalculation on every frame
- Too many edges rendered simultaneously
- Heavy post-processing at high resolution
- Particle counts too high on low GPUs

Mitigations

- **Instanced rendering** for nodes/particles
- **Single geometry buffer** for edges
- **Throttle layout** to “settle,” then freeze positions
- **Dynamic resolution** scaling:
  - drop render resolution to 0.75x under load
- **LOD caps** with strict thresholds

Virtualization

At LOCAL tier, do not render entire world.
Render:

- active neighborhood
- relevant clusters
- contextual area around focus point

---

14. Accessibility Contract

- Min hit target: `44px` (tap/selection radius, even if visual node is smaller)
- Keyboard navigation (web):
  - tab cycles through search results rather than 100k nodes
- Reduced motion:
  - disable drift + breathing
  - reduce bloom intensity
  - shorten camera transition to 150–250ms
- Contrast:
  - glow must not wash out labels (when labels present)
- Screen reader:
  - WebGL canvas is decorative; focusable HTML list/search provides semantic access

---

15. Edge Case Mapping

- Empty graph: show single node “seed” + gentle glow, prompt to add
- Very dense graph: auto switch to CLUSTER tier + prompt “zoom in for detail”
- Loading:
  - progressive reveal:
    1. background
    2. clusters
    3. nodes
    4. edges
- API failure:
  - cached last-known state + toast + retry
- Orientation resize:
  - preserve camera framing with aspect correction

---

16. Analytics Contract

Track interaction as product signals (this visualization will be your moat).

Events

- `graph_zoom_changed`
  - `{ zoomLevel, lodTier, durationMs }` (debounce 300ms)
- `node_hovered`
  - `{ nodeId, nodeType, clusterId }` (throttle 500ms)
- `node_selected`
  - `{ nodeId, nodeType, clusterId, zoomLevel }`
- `cluster_selected`
  - `{ clusterId, memberCount }`
- `filter_applied`
  - `{ filters, resultsCount }`
- `drill_in`
  - `{ fromTier, toTier, targetId }`

Haptics/sound: none (web); if mobile later, light haptic on select.

---

17. Acceptance Criteria (Concept Fidelity)

“Wow” must be true

- Zooming out transforms representation (not just scaling)
- Universe tier feels alive (subtle drift + nebula)
- No hairball at any zoom level
- Selection feels cinematic (smooth focus + neighbor illumination)
- 60fps on Apple Silicon with 10k nodes / 50k edges in LOCAL neighborhood mode

“Clarity” must be true

- At LOCAL tier, a user can immediately see “what connects to what”
- At CLUSTER tier, a user can identify communities and bridges
- At UNIVERSE tier, a user can see “where the action is” (major regions)

---

18. Suggested Build Plan (Fastest Path to Your Concept)

Phase A — Prototype the Renderer (1–2 weeks)

- WebGL scene with:
    - instanced node sprites + glow shader
    - curved edges + gradient
    - bloom pass
- Basic camera zoom + LOD switching (manual thresholds)

Phase B — Cluster + Hull (1–2 weeks)

- Community detection + centroid calc
- Hull generation + soft glow
- Crossfade LOCAL <-> CLUSTER

Phase C — Universe Mode (1–2 weeks)

- Nebula particles per cluster
- Inter-cluster streams
- Drift field (optional reduced motion support)

Phase D — Polish (ongoing)

- GPU picking
- Analytics
- Performance scaling + resolution drops
- Editor/debug panels for tuning tokens

---

19. Developer “Beauty Layer” Checklist (Non-Negotiables)

If any of these are missing, the output will look like a generic graph tool.

- Additive blending enabled for glow layers
- Bloom post-process tuned (not nuclear)
- Curved edges with gradients (no straight spaghetti)
- Pulse motion on key edges
- Cluster hulls with soft fog
- Universe mode with nebula particles + streams
- LOD crossfade transitions (not hard cuts)
- Edge density caps + neighborhood rendering

---

20. Validation Loop (Internal Gate Before Shipping)

Validation Pass

- Every animation has duration/easing
- Every LOD has render rules + caps
- Picking works without frame drops
- Reduced motion is respected
- Edge count never exceeds caps

Efficiency Audit

- No per-edge CPU animation
- No re-layout every frame
- No heavy third-party graph renderer that blocks art direction

Regression Risks

- Zoom tier desync -> mitigate with explicit LOD manager + hysteresis
- Hairball return -> enforce caps in code, not “by convention”
- Perf drop on huge graphs -> neighborhood render only

Deliverables to Hand Your Team Today

1. **Rendering tokens file** (colors/glow/motion/lod thresholds)
2. **Three LOD specs** (what renders + caps)
3. **Shader requirements** (node glow, edge gradient, dash pulse)
4. **Cluster pipeline** (community detect + hull + centroid)
5. **Acceptance test scenario** (10k/50k stress test, measure FPS)

Onboarding

You’re right to focus on this early. If onboarding feels like **data entry**, the product dies.

North Star onboarding must feel like **discovering yourself**, not filling out a profile.

The mental shift is:

People should not build their constellation.
The constellation should emerge from their life.

Everything about onboarding should reinforce that.

note: plenty of this will be manual entry in the first node, but ai automation and auto parsing/ reasoning is the first thing I want to work on once the MFP is done

---

The Core Principle

Onboarding must follow this rule:

“Never ask the user to type something the system could infer.”

Instead the system should:

1. ingest artifacts
2. extract signals
3. build the constellation automatically
4. let the user refine it

The user becomes an **editor**, not a data clerk.

The Ideal Onboarding Feeling

The user should feel:

> “Wait… it built this from my stuff?”

That surprise is the hook.

Step 1 — Identity Anchor (30 seconds)

Start with a single prompt:

“What do you spend most of your time building or doing?”

Examples appear below:

- software
- music
- restaurants
- architecture
- marine biology
- welding
- comedy
- skateboarding

This establishes their **primary domain cluster**.

Immediately show:

their first node forming on the screen.

The constellation begins.

Reward loop begins immediately.

Step 2 — Artifact Upload (The Magic Step)

Instead of asking questions, ask for **things they already have**.

Prompt:

“Drop anything that shows your work.”

Accept:

- resume
- portfolio
- GitHub
- LinkedIn
- personal website

- PDFs
- articles
- research papers
- videos
- social accounts
- photos
- Presentations
- voice

Then run an **AI parser** that extracts:

Projects
Skills
Decisions
Domains
Achievements

The constellation grows automatically.

Nodes appear in real time.

This moment should feel like **watching a galaxy form.**

Step 3 — Timeline Builder (Low Effort)

Ask one simple question:

“What are 3 things you’ve worked on that mattered to you?”

As they type:

Nodes appear.

Edges connect.

Each project generates:

- skills
- decisions
- outcomes

Users are not filling fields — they are **telling stories.**

Step 4 — Relationship Signals

Ask:

“Who influenced how you think?”

Examples appear:

- mentors
- collaborators
- teachers
- inspirations

Each entry creates:

Influence edges.

This is where constellations begin to feel alive.

Step 5 — The First Gravity Boost

Once a few nodes exist, show:

Constellation Score

Example metrics:

- capability density
- project depth
- domain clarity
- influence edges

Visual reward:

Nodes glow brighter.

Connections animate.

Clusters form.

Users immediately want to improve it.

The Psychological Trick

People love completing systems.

Especially when they **see progress visually.**

You want the same feeling as:

- leveling up a character
- building a civilization map
- completing a skill tree

But with their real life.

Constellation Growth Mechanics

As users add information, the constellation becomes:

More colorful
More connected
More dense

Visual effects:

New connections appear as:

particle trails

Clusters appear as:

nebula clouds

Nodes evolve visually as they gain gravity.

This becomes the **reward loop.**

Progressive Onboarding

Never ask everything up front.

Instead unlock prompts over time.

Example prompts:

“Add a decision that changed how you work.”

“Add someone who influenced your thinking.”

“Add a project that failed but taught you something.”

Each adds new graph depth.

The Feedback Moment

After onboarding show them:

“Your Capability Map.”

Then highlight patterns:

Example:

> Your work clusters around systems thinking and marketplaces.
>
> Your decisions show strong bias toward experimentation.
>
> Your strongest domain is AI-enabled services.

This creates **intellectual curiosity.**

People want to refine the map.

The Shareable Payoff

Immediately after onboarding:

Generate a **public constellation view.**

Example share:

Explore my capability constellation.

People are naturally curious.

This drives growth.

The Hidden Social Incentive

When two users join:

Edges appear between them.

Example:

Shared projects
Similar domains
Mutual collaborators

This creates:

network gravity.

The Long-Term Hook

Over time North Star becomes:

the most interesting representation of a person online.

Not a bio.

A **thinking map.**

The Real Reward

The true payoff is not visuals.

It is **recognition.**

When people explore someone's constellation and say:

> "Wow… this person is fascinating."

That's the addictive loop.

The Real Onboarding KPI

Your onboarding goal is simple:

User should see **their constellation appear within 60 seconds.**

If that happens:

They continue.

If onboarding requires 20 minutes of typing:

They leave.

The Three Design Rules

North Star onboarding must always:

1. **Extract more than it asks**
2. **Visualize instantly**
3. **Reward every contribution**

The Future Moment

Eventually the best onboarding will be:

User signs in.

North Star asks:

“Want me to build your constellation?”

They connect:

- email
- portfolio
- GitHub
- writing
- social

And the constellation appears automatically.

That is the moment the product becomes magical.

Graph Expansion Mechanism

Alright. Here's the graph expansion mechanism that keeps your universe **high-signal, compounding, and not polluted**.

This is a system design, not a vibe.

North Star Graph Expansion Mechanism

Goal

Every new user and every new action must do **two things**:

1. **Help the user locate themselves** in the universe fast
2. **Increase the map's accuracy** for everyone else

If it doesn't do both, it's noise.

---

1) The Prime Directive: "Claims don't create stars. Evidence does."

The universe collapses into BS if you let people type anything.

So we separate:

- **Capability Claims** (cheap)
- **Capability Signals** (expensive / verified)

Only signals create strong nodes + strong edges.

Claims can exist, but they have low gravitational weight until validated.

---

2) The Expansion Loop (What happens when a new user joins)

Step 0 — Identity Node (quietly created)

A person node exists, but it's not the center. It's a container.

Person node starts dim.

Step 1 — The "3 Capabilities" seed (required)

User adds **3 capabilities they can actually do**.

But you don't let them free-type at first.

Selection UX

- Search + pick from **existing capability nodes** (preferred)
- If not found: "Propose new capability" (quarantined, not live)

This prevents taxonomy explosion.

Step 2 — Evidence attachment (lightweight, not annoying)

For each capability, user chooses 1 evidence type:

- Project (link or short description)
- Portfolio artifact
- Repo
- Work sample
- Credential
- Peer confirmation (later)
- "No evidence yet" (allowed but flagged as low weight)

This drives signal quality without heavy verification overhead on day one.

Step 3 — Auto-connect (system suggests edges)

System proposes:

- Similar capabilities
- Adjacent capabilities (often co-occurring)
- Potential clusters (“you’re in Design Ops / Dental Revenue / etc.”)

User confirms or rejects a few suggestions.

That single action trains edge confidence fast.

---

3) How new nodes are allowed into the universe (Gatekeeping without friction)

Node Types and Rules

A) Capability Nodes

Creation rule:

- New capability starts in **Proposed state**
- It only becomes a “real star” when it hits thresholds:

Threshold example:

- ≥ 5 unique users attach it
- ≥ 2 users attach evidence
- It maps cleanly into an existing domain cluster

Until then: it’s dim, not central, not forming clusters.

B) Project Nodes (high leverage)

Projects are a major source of truth because they naturally connect:

- people
- capabilities
- organizations
- outcomes

Project nodes are your best expansion fuel because they create multiple edges at once.

Creation rule:

- A project node can be private or public
- Public projects strengthen the universe faster

C) Concept Nodes (controlled vocabulary)

Concepts (e.g., “Revenue Operations”, “Lactation Support”, “LLM agents”) should not be free-typed by everyone or you’ll get chaos.

Creation rule:

- Concepts mostly come from:
  - seeded ontology
  - admin curation
  - or high-confidence emergent clusters promoted into concepts

---

4) Edge Creation Rules (this is where graphs succeed or die)

Edges should not be “user says so.”

Edges should be formed from **repeatable co-occurrence** and **actions**.

Edge Types (minimum viable set)

1. `PERSON_HAS_CAPABILITY` (weak until evidence)
2. `CAPABILITY_CO_OCCURS_WITH_CAPABILITY` (statistical)
3. `CAPABILITY_USED_IN_PROJECT`
4. `PEOPLE_COLLABORATED_ON_PROJECT`
5. `CAPABILITY_SIMILAR_TO_CAPABILITY` (ML embedding + confirmation)

Edge Weight Sources (ranked)

Highest to lowest:

1. Shared project evidence (strong)
2. Peer confirmation (strong, later)
3. Repeated co-occurrence across many users (strong-ish)
4. ML similarity only (weak)

5. Self-claim only (weakest)

Rule: ML similarity never creates a high-weight edge without human confirmation or repeated evidence.

---

5) Preventing Pollution (anti-spam + anti-ego + anti-keyword gaming)

A) Quarantine everything new

New capability proposals:

- exist
- but don't reshape the universe until validated

B) Rate limits

- Max new proposed capabilities/day per user
- Max edits/day without evidence

C) Duplicate detection

Before creating a new capability node:

- run fuzzy match + embedding similarity against existing capability nodes
- offer merge suggestions

D) "Gravity tax" for vague skills

"Leadership", "Hard worker", "Communication" can exist, but they can't become galactic hubs unless backed by:

- outcomes
- projects
- endorsements
- repeated contextual pairing

Otherwise your universe becomes LinkedIn.

---

6) The Compounding Flywheel (how the universe grows fast without manual moderation)

Flywheel A — Capability Attachment

More users attach to existing capabilities → those nodes get stronger → better clustering → better recommendations → more users.

Flywheel B — Project Anchors

Projects create dense edge bundles → makes clusters stable → makes bridges visible → increases discovery value.

Flywheel C — Bridge Discovery

When the engine detects a new bridge (capability connecting two clusters), it prompts:

- “This looks like a bridge skill. Confirm?”
- “Add a project that proves it?”

Bridges are the highest-value nodes in the system. Treat them like treasure.

---

7) “Universe Growth” by Design: Progressive Rights

Not everyone gets to reshape the ontology on day one.

Trust Level 0 (new)

- attach to existing capability nodes
- propose new nodes (quarantined)

Trust Level 1 (signal contributor)

- can promote/merge proposals
- can confirm similarity edges

Trust Level 2 (curator)

- can approve new capability nodes into canonical set
- can define aliases and taxonomy mapping

This keeps the map clean.

---

8) Time: Decay + Seasonality (prevents stale universes)

Capability relevance shifts.

So edges and node gravity must incorporate **recency**.

Decay model

- edges lose weight slowly if not reinforced
- capabilities can “cool down”
- clusters can fade or rise

This keeps the universe alive and prevents 2012 knowledge from dominating 2026.

---

9) How clustering updates safely (no jarring re-layout chaos)

You don’t want the whole universe to rearrange constantly.

Approach

- LOCAL tier: layout is responsive and can adjust quickly
- CLUSTER tier: cluster positions update in batches (daily or weekly)
- UNIVERSE tier: only updates on major shifts; keep camera landmarks stable

Rule: preserve mental map. Users hate when their universe “teleports.”

---

10) The Expansion MVP (what you ship first)

If you build all of this at once, you’ll stall.

Ship this core:

1. Seed capability ontology (10k-ish)
2. User attaches 3 capabilities
3. Evidence optional but strongly encouraged
4. Edges formed by co-occurrence + confirmation
5. New capability proposals quarantined + require thresholds
6. LOD rendering: local / cluster / universe

That’s enough to start compounding.

---

11) Acceptance Criteria (expansion quality gates)

The universe is growing correctly if:

- New users mostly attach to existing capabilities (not invent new ones)
- Proposed capabilities merge rate is high (duplicates handled)
- Clusters become more coherent over time (higher modularity score)
- Bridge nodes increase discovery actions (click-through, follow, contact, “learn next”)
- The “graph feels smarter” after every 100 new users

If you see:

- explosion of near-duplicate skills

- clusters that don’t stabilize
- everything connecting to everything
  you’ve lost the ontology war.

Gravity Engine V1 (CTO)

North Star

Gravity Engine v1

CTO Instruction Version — Atomic Unit First

This is the scoring and propagation system for the **one-node MFP**.

The job of Gravity Engine v1 is not to crown “winners.”
Its job is to make the constellation feel:

- legible
- alive
- earned
- expandable

For v1, gravity must answer one question well:

> What parts of this person’s capability graph matter most, and why?

If that works for one profile, it can scale to many.

For the one-node MFP, keep the live formula simple and explicit.

Use:

- fixed weighted formulas
- editorial override
- cached recompute

Do not use ML ranking yet.

You need trust, legibility, and speed first.

---

1. Purpose

Gravity is the system that determines:

- which nodes shine brighter

- which nodes pull attention first
- which clusters feel dense and important
- which relationships appear strongest
- how new information changes the shape of the constellation

In v1, gravity is **not social popularity**.

It is a structured representation of:

- demonstrated depth
- relationship density
- proof density
- recency of active work
- narrative importance

---

2. Design Rules

Rule 1 — No vanity metrics

Do not use:

- followers
- likes
- impressions
- views as primary signal

Those can destroy trust.

Rule 2 — Capability over status

A chef, welder, scientist, founder, or comic should all be scoreable within their own domain using the same structural logic.

Rule 3 — Evidence beats claims

A node with supporting artifacts, outcomes, and linked decisions should carry more gravity than a vague self-description.

Rule 4 — Local meaning first

For v1, gravity only needs to make sense **within one person's graph**.
Global cross-network ranking comes later.

Rule 5 — Stable, not twitchy

Gravity should not fluctuate wildly from tiny edits.
It should feel alive, but calm.

---

3. What Gravity Controls in the UI

Gravity is not just a number. It drives rendering.

Visual effects driven by gravity

- node size
- glow intensity
- edge brightness
- edge thickness
- cluster density feel
- orbit priority
- default camera emphasis

Interaction effects driven by gravity

- which nodes appear first on initial load
- which nodes stay visible at wider zoom levels
- which nodes are surfaced in summaries
- which nodes are suggested in “Ask” follow-ups

---

4. Gravity Objects to Score

In v1, gravity applies to these node types:

- Profile
- Project
- Decision
- Skill
- Outcome
- Idea (optional, low priority in v1)

Each node type gets a different scoring formula.

---

5. Core Inputs

These are the only inputs Gravity Engine v1 should use.

A. Structural inputs

- number of direct connections
- number of second-order meaningful connections
- relationship types
- cluster membership

B. Evidence inputs

- citations attached
- artifacts attached
- explicit outcomes attached
- documented decisions attached

C. Content inputs

- completeness of node
- clarity of node description
- presence of measurable metrics
- failure log presence
- next-proof presence

D. Time inputs

- recency of updates
- active project status
- recent additions to a cluster

E. Manual curation inputs

For v1 only, allow explicit weighting override for founder graph curation.

That lets you shape the first experience while learning.

---

6. Scoring Model by Node Type

Profile Gravity

The profile node should be strongest by default because it anchors the constellation.

Formula v1

```
profile_gravity = base + project_depth + evidence_density +
cluster_diversity + activity
```

Components

- `base = 100`
- `project_depth = sum(project gravity * 0.35)`
- `evidence_density = total cited artifacts * 2`
- `cluster_diversity = number of distinct meaningful project clusters * 5`
- `activity = recent update bonus capped`

Purpose

Ensures the profile remains the center star without overwhelming everything else.

Project Gravity

Project nodes are the most important functional units after the profile.

Formula v1

```
project_gravity = decision_weight + outcome_weight + evidence_weight +
structure_weight + recency_weight
```

Suggested weights

- documented decisions: `+8 each`
- documented outcomes: `+10 each`
- attached artifacts/citations: `+4 each`
- system design present: `+10`
- metrics section present: `+8`
- failure log present: `+8`
- next proof present: `+6`
- active project status: `+10`

- recent update within 30 days: `+5`

Purpose

Projects with richer evidence and clearer thinking should visibly matter more.

Decision Gravity

Decisions are one of North Star's core differentiators.

Formula v1

```
decision_gravity = centrality + consequence + support +
reversibility_bonus
```

Components

- connected to project: required
- linked outcome present: `+8`
- linked failure or tradeoff present: `+8`
- reversal trigger present: `+6`
- cited artifact support: `+4 each`
- reused across multiple projects later: future multiplier

Purpose

Show that consequential decisions carry real weight.

Skill Gravity

Skills should not dominate visually in v1.

They are supporting nodes, not identity theater.

Formula v1

```
skill_gravity = usage_count + project_diversity + evidence_support
```

Components

- connected projects count
- connected outcomes count
- explicit evidence from artifacts or decisions

Purpose

Skills gain gravity when demonstrated repeatedly, not just listed.

Outcome Gravity

Outcomes matter because they prove consequence.

Formula v1

```
outcome_gravity = measurability + project_importance + proof_strength
```

Components

- measurable numeric result present: `+10`
- clearly stated qualitative outcome: `+5`
- connected project gravity multiplier
- evidence or artifact support: `+4 each`

Purpose

Outcomes should feel like anchor points of consequence.

Idea Gravity

Optional in v1.
Keep weak unless tied to projects or decisions.

Formula v1

```
idea_gravity = linkage + relevance + support
```

If an idea is not tied to action, keep it dim.

---

7. Edge Weighting

Edges also need gravity-like weighting.

This determines brightness and visual importance.

Edge types and base weights

- Profile → Project: `1.0`
- Project → Decision: `0.9`
- Project → Outcome: `0.95`
- Project → Skill: `0.65`
- Decision → Outcome: `0.85`
- Decision → Skill: `0.6`
- Idea → Project: `0.5`

Boosters

Add to edge weight if:

- edge supported by evidence
- edge part of a currently active cluster
- edge touched by recent updates

Penalties

Reduce weight if:

- weakly supported
- inferred but not confirmed
- stale and disconnected

---

8. Visual Translation Layer

Translate gravity score to rendering behavior.

Node size

Use a capped logarithmic scale.
Do not let one node become absurdly large.

Glow intensity

Higher gravity = brighter core and slightly wider halo.

Label visibility

At wider zoom levels, only higher gravity nodes keep labels.

Edge intensity

High-weight edges render:

- brighter
- slightly thicker
- with more visible pulse on selection

Cluster aura

Cluster aura intensity = aggregate project + decision gravity within cluster.

---

9. Initial Rendering Rules

When the profile first loads, do not show everything equally.

The engine should emphasize:

1. profile node
2. top 3 project nodes
3. top 2–4 strongest decision or outcome nodes per project

This creates instant structure and avoids visual mud.

---

10. Gravity States

Each node should have one of four visual states.

Dormant

Low gravity. Visible but dim.

Active

Normal node with readable presence.

Significant

High-value node. Bright, clear halo, visible label.

Anchor

Major node. Used sparingly.
Usually profile + strongest projects.

This prevents the graph from feeling flat.

---

11. Temporal Behavior

Gravity should include time, but carefully.

Recency bonus

Use only small bonuses for:

- recent edits
- current project activity
- newly added evidence

Decay

Do not aggressively decay old important work.
Good work should stay meaningful.

Use:

- soft decay on uncited, unmaintained low-importance nodes
- no major decay on strong anchor projects

---

12. Founder Override Layer

For the atomic unit, you need some human editorial control.

Add optional fields:

- `gravity_override`
- `featured`
- `suppress_from_initial_view`

Use this sparingly.

Purpose:

- tune the first experience
- avoid awkward auto-weighting mistakes
- make the MFP feel intentional

---

13. Data Fields Needed

Add these to node records or a companion scoring table:

- `base_score`
- `computed_gravity`
- `gravity_state`
- `evidence_count`
- `citation_count`
- `connected_projects_count`
- `connected_outcomes_count`
- `last_activity_at`
- `gravity_override`
- `is_featured`

For edges:

- `base_weight`
- `computed_weight`
- `support_count`
- `last_activity_at`

---

14. Compute Strategy

For v1, do not over-engineer this.

Recommendation

Use asynchronous score computation on content changes.

Trigger recompute when:

- node created
- node edited
- edge added
- evidence attached
- outcome added
- project status changed

Pipeline

1. content saved
2. affected subgraph identified
3. recompute local gravity only
4. cache updated values
5. client fetches precomputed values

Do not compute full graph physics on every request.

---

15. API Requirements

Get graph with gravity

`GET /profile/:id/graph`

Returns:

- nodes with `computed_gravity`, `gravity_state`
- edges with `computed_weight`

Recompute gravity

`POST /profile/:id/recompute-gravity`

For admin/editorial use in v1.

Node detail

`GET /node/:id`

Returns:

- metadata
- supporting artifacts
- why this node has its gravity

That last part matters.

You should be able to explain:

> why is this node bright?

---

16. Explainability Requirement

Every gravity score must be decomposable.

For each node, store a simple explanation object:

```
{
  "top_factors": [
    "3 documented decisions",
    "2 linked outcomes",
    "5 supporting citations",
    "active project"
  ]
}
```

This helps:

- debugging
- trust
- future user-facing explainability

---

17. Acceptance Criteria

Gravity Engine v1 is done when:

Visual

- the graph no longer feels flat
- important nodes are obvious without being loud
- the profile feels like a structured universe, not random dots

Cognitive

- a new visitor can tell what matters in under 10 seconds
- project importance feels intuitively correct
- decisions and outcomes feel connected, not decorative

Technical

- recompute finishes fast for one profile
- rendering remains smooth at v0 node counts
- no visible score jitter from small edits

Product

- users spend more time exploring high-gravity nodes
- node emphasis feels earned
- the graph suggests depth and future scalability

---

18. What Not to Do

Do not:

- rank by social popularity
- overweight recent edits
- make gravity mysterious and unexplainable
- let skills outrank projects without evidence
- let visual spectacle override semantic meaning

If gravity feels arbitrary, trust dies.

---

19. v1 Success Metric

Gravity Engine v1 succeeds if someone explores the graph and naturally understands:

- what this person is best known for
- which projects matter most
- which decisions shaped their work
- where the real proof lives

Without being told.

---

20. Strategic Role

This system is the beginning of North Star's moat.

Today it powers:

- one constellation

Later it powers:

- discovery
- ranking
- search relevance
- cross-profile recommendation
- capability clustering

This is where North Star starts becoming more than a beautiful interface.

It becomes a **capability engine**.

The next thing your CTO will need after this is the **actual implementation package**:

1. database fields and migration spec
2. TypeScript scoring functions
3. render mapping rules
4. recompute job triggers
5. admin tuning interface

If you want, I'll write that package next in code-facing format.

North Star Physics

Good. Now we move to the **physics of the universe**.

Without a physics model, your graph becomes what almost every graph product becomes:

> a glowing hairball.

To avoid that, the system must behave like a **real universe**.
Things must attract, repel, cluster, and orbit according to rules.

That's what gives the graph **structure and meaning**.

The Physics of the North Star Universe

In the real cosmos, structure forms because of **gravity**.

In your system, structure forms because of **capability gravity**.

This determines:

- which nodes move together
- which clusters form
- which bridges appear
- what becomes a hub
- what fades away

Everything your renderer shows is the **visual consequence of this physics layer**.

The Core Principle

Not all capabilities are equal.

Some capabilities:

- connect many domains
- appear across many industries
- enable other capabilities

Those capabilities create **gravitational pull**.

Capability Gravity

Each capability node receives a **gravity score**.

Example factors:

Gravity Score =
connection_count
+ cross_domain_connections
+ collaborator_count
+ usage_frequency
+ evidence_weight

High gravity capabilities pull other nodes toward them in layout space.

Example:

Machine Learning

connects to:

medicine
finance
robotics
climate
biology

So it becomes a **galactic hub**.

Three Forces That Shape the Graph

Your system needs three forces:

1. Attraction

Capabilities that frequently appear together pull toward each other.

Example:

React
↔
TypeScript
↔
Node.js

This creates clusters.

Implementation concept:

edge_weight → attraction_force

---

2. Repulsion

Nodes must repel to prevent collapse.

Otherwise the graph becomes a pile.

Implementation:

node_repulsion ∝ node_degree

High-degree nodes repel more strongly to maintain spacing.

---

3. Gravity (Importance)

Important nodes pull nearby nodes closer.

This creates **hubs**.

Implementation:

gravity ∝ capability_importance

This ensures the system visually reveals:

- hubs
- bridges

- specialists

Cluster Formation

Clusters emerge automatically from capability relationships.

Example cluster:

Computer Vision
Deep Learning
Image Segmentation
OpenCV
PyTorch

Your cluster engine detects these with algorithms like:

- Louvain
- Leiden

The cluster then becomes a **constellation**.

Bridge Detection (The Magic Layer)

Some nodes connect clusters.

Example:

Bioinformatics

connects:

Biology
↔
Machine Learning

These nodes are **bridges**.

Bridges are the most important discoveries in the system.

They should be visually emphasized.

Rendering suggestions:

- thicker glow
- animated orbit lines
- unique color halo

Capability Evolution

Capabilities are not static.

They change over time.

Your system should track:

growth_rate
collaboration_rate
emerging_connections

This lets the graph show:

- **rising clusters**
- **declining clusters**
- **new constellations forming**

Visually:

Emerging clusters glow brighter.

Multi-Scale Physics

The same rules operate at all zoom levels.

Local scale:

capability ↔ capability

Mid scale:

cluster ↔ cluster

Universe scale:

domain ↔ domain

This consistency makes the system feel **coherent and natural**.

The Three Types of Stars

To make the map readable, nodes should visually reflect their role.

1. Hubs

High gravity nodes.

Examples:

JavaScript
Marketing
Project Management

Visual style:

- larger glow
- brighter center

---

2. Bridges

Nodes connecting clusters.

Examples:

AI Ethics
Data Journalism
Computational Biology

Visual style:

- dual color gradient
- pulsing halo

---

3. Specialists

Nodes deep in clusters.

Examples:

finite element modeling
prosthetic gait analysis
react fiber reconciliation

Visual style:

- smaller
- dimmer
- tightly grouped

The Layout Engine

The layout algorithm should combine:

ForceAtlas2
+
hierarchical clustering
+
importance weighting

Basic formula concept:

Force = attraction + repulsion + gravity

with edge weights and node gravity influencing strength.

Preventing Graph Chaos

Without constraints, large graphs become unreadable.

You must enforce:

Edge limits

Only render meaningful edges above a threshold.

Neighborhood rendering

Local zoom shows only relevant neighborhood.

Cluster aggregation

Mid zoom collapses nodes into clusters.

Universe abstraction

Far zoom shows domains instead of nodes.

How This Supports Your Visualization

When physics works correctly:

Zoomed in:

skills + collaborators

Zoomed out:

communities

Zoomed way out:

fields of knowledge

The beauty layer simply reveals this structure.

The Real Outcome

When the physics layer works, your graph answers questions instantly.

A user can see:

Where are the hubs?
Where are the bridges?
Where is innovation happening?
Where do I fit?

This is why the system becomes powerful.

It shows **structure humans can’t see in lists or search results**.

The Final System Model

Your engine becomes three stacked layers:

Layer 1

Capability Physics Engine

Determines structure.

Layer 2

Graph Rendering Engine

Displays structure.

Layer 3

Exploration Interface

Lets humans navigate the universe.

One Final Thought

Most graph products fail because they start with visualization.

You are doing the right thing by starting with:

why
→ atom
→ physics
→ rendering

That order is what will make the system feel **inevitable instead of artificial**.

Final Build Ledger Pages

PART I — Final Build Ledger Pages (first node: Prentiss) (MDX)

Below is production-grade copy. Calm. Minimal. No hype. Will be used to help with onboarding.

🔵 GetIT — Coordination Layer for Services

One-Liner

A video-first coordination layer that converts trust into booked outcomes — designed to become ambient-native infrastructure.

System Design (Plain English)

GetIT runs a video-first local services feed that converts trust into transactions.

It layers:

- A trust surface (short-form service videos)
- A context layer (time, location, behavior)
- Modular agents that route discovery → intent → booking → payment

The goal is not browsing.
The goal is resolved intent.

Constraints

- Liquidity per square mile
- Trust formation speed

- Vendor density
- Fraud / payment disputes
- Intent-to-book friction
- Ambient-native compatibility

Core Decisions

Marketplace vs Coordination Layer

Chose coordination layer.
Reframed the product from listings to intent resolution.

Ads vs Density Loops

Chose micro-area density over paid sprawl.
Houston Heights wedge.
Cluster-first growth.

Screen App vs Ambient OS

Architected early for voice intent parsing, context layering, and modular agents.

Failure Log

Engineering sophistication outpaced market validation.

Built:

- CapCut-grade editor
- AI tagging
- Federated personalization
- Agent modularization

Before proving repeat booking velocity inside a single dense vertical.

Lesson:
Density > intelligence early.

Metrics That Matter

1. Booking density per square mile
2. Repeat booking rate (30/60/90 day)
3. Time-to-intent fulfillment

If those win, ambient expansion becomes credible.

2-Week Next Proof

Run a density experiment in one Houston micro-area:

- 15–25 vendors
- 2–3 service categories
- Measure real booking velocity + repeat behavior

If bookings don't compound locally, simplify the funnel until they do.

🟢 Fast Food — Decision Compression Engine

One-Liner

An operating system that collapses "what's for dinner?" into a sub-3-minute decision for cognitively exhausted households.

System Design (Plain English)

Fast Food collects lightweight context signals:

- Time
- Weather
- Preferences

- Household patterns

It runs a local-first decision engine that compresses dinner into one clear recommendation.

The UX is calm, immersive, and intentionally narrow.

No browsing.
No endless choice.
One answer.

Constraints

- Cognitive fatigue
- Habit formation
- Privacy expectations
- Offline resilience
- Context accuracy
- Minimal UI tolerance

Core Decisions

Recipe App vs Decision Compression

Chose compression.
Measure time-to-decision, not recipe views.

Cloud AI vs Local-First

Chose local-first inference + Supabase sync.
Optimize speed, privacy, resilience.

Feature Expansion vs Living Canvas

Chose immersive minimalism over feature creep.

Failure Log

Vision clarity outpaced real user loop validation.

No measured proof yet that:

- Sub-3-minute decisions become weekly habit
- Suggestion acceptance compounds

Lesson:
Test behavior before refining aesthetics.

Metrics That Matter

1. Time to decision (<3 minutes)
2. Weekly active households
3. Decision acceptance rate

2-Week Next Proof

Run a 14-day exhausted-household test:

- 20 families
- Track decision time, acceptance rate, repeat usage

If overrides are high, tighten logic and reduce choice width.

🟣 Anansi — Knowledge Acquisition OS

One-Liner

An AI-native lesson-to-game system that converts standards into playable, Chromebook-friendly indie-grade quests with measurable mastery reporting.

System Design (Plain English)

Anansi converts standards-aligned lessons into playable quests using an AI lesson-to-game generator.

It outputs:

- Browser-based indie-grade experiences
- Chromebook compatibility
- Mastery capture data
- Credential-ready reports for teachers and parents

The goal is not engagement.
The goal is measurable cognitive lift.

Constraints

- Standards alignment (TEKS / state-aligned frameworks)
- Chromebook hardware limits
- Classroom time budgets
- Teacher reporting needs
- Engagement vs rigor balance

Core Decisions

Content Platform vs Knowledge OS

Chose operating system — generation + measurement loop.

Gamification vs Judgment Training

Chose cognitive durability over surface engagement.

Broad Audience vs Narrow Wedge

Chose one standards unit pilot before expansion.

Failure Log

Abstraction initially outpaced wedge clarity.

Lesson:
Every OS needs a painfully concrete entry point.

Metrics That Matter

1. Completion rate
2. Replay rate
3. Pre/post mastery lift

2-Week Next Proof

Pilot one TEKS-aligned standards unit:

- Generate 5–10 playable quests
- Test with 10–15 students
- Measure completion, replay, mastery lift

If rigor or engagement breaks, fix the generator before expanding scope.

PART IV — High-End WebGL (Only When Earned)

When we add WebGL:

For GetIT page:

Subtle pulsing density field — representing micro-area clusters.

For Fast Food:

Soft animated gradient that shifts with time-of-day.

For Anansi:

Particle nodes forming structured networks (knowledge graph aesthetic).

But this comes after traction of the core surface.

Final Observation

Your pattern is clear:

You are an infrastructure-biased operator who thinks in operating systems, not apps.

Your site must feel like an operating system.

Calm. Structured. Testable.

Next move:

Will also need to draft:

1. Your **Start Here** page (fully written, calm minimal)
2. The **Ask Me Anything** system prompt wired to your actual voice
3. The JSON schema for project ingestion (clean, future-proof)
4. The exact Next.js folder structure so dev can begin tomorrow

Auto-Generation Logic

PART II — Auto-Generation Logic (Platform Thinking)

Now we formalize the automation.

When a user uploads files + certifies their project, the LLM should auto-generate:

- One-liner
- System Design (plain English)
- Constraints
- 3 Core Decisions
- Failure Log
- Metrics
- 2-week proof

This becomes frictionless.

Upload → Structured Extraction Flow

Step 1: Parse

Extract:

- Project name
- Architecture mentions
- Roadmap sections
- “Decision” language
- Tradeoffs
- Metrics references
- Postmortems
- Planning docs

Step 2: Normalize

Convert messy language into this schema:

```
Project {
  one_liner
  system_design_plain
```

```
  constraints[]
  decisions[] {
      option_a
      option_b
      chosen
      why
  }
  failure {
      what_broke
      lesson
  }
  metrics[]
  two_week_proof
}
```

Step 3: Confidence Pass

LLM checks:

- Are decisions explicit?
- Are metrics measurable?
- Is 2-week proof testable?

If vague → prompt user:

> “I see vision depth but no measurable proof. Would you like help defining a 2-week experiment?”

This keeps friction low but standards high.

PART III — 80/20 Platform Smarts (Later Within Scope)

No massive features. Just leverage.

🔹 1) “Decision Extraction Mode”

After upload, the system highlights:

> “We detected 4 decision tradeoffs. Confirm?”

User clicks confirm → done.

That alone makes onboarding feel magical.

🔹 2) “Shadow Pattern Detection”

After 3+ projects uploaded, system auto-generates:

> Pattern Across Projects:
> You consistently choose infrastructure over surface features.
> Risk: horizon bias.

That’s powerful and differentiating.

🔹 3) Automatic 2-Week Proof Generator

After profile creation:

> “Would you like the system to generate a 2-week proof test for each project?”

Yes → It outputs exactly what you wrote above.

This makes everyone look operational.

🔹 4) Calm UX Principle for Onboarding

No “complete your profile.”

Instead:

- Step 1: Upload artifacts
- Step 2: Confirm extracted decisions

- Step 3: Approve 2-week proof
- Step 4: Publish

Under 10 minutes.

That’s the magic.

Next.js

Here are the 4 deliverables, ready to hand to a dev and start building immediately for the MFP first node. Will serve as the template for all nodes later.

---

1) Start Here Page (MDX, calm + minimal)

Save as: `content/pages/start-here.mdx`

```
---
title: "Start Here"
description: "A calm, evidence-backed interface to evaluate my work."
---

# Prentiss

Frontier operator. I build adoption-ready systems at the edge of AI.

This is not a resume. It’s a proof surface.

If you’re evaluating me, I want you to see how I think:
- what I chose
- what I didn’t build
- what broke
- what I measure
- what I would do next

---

## How to use this site (60 seconds)

1) **Run a Fit Check**
Paste a role brief or problem statement. You’ll get a structured fit verdict with evidence links.

2) **Ask about my work**
Interrogate any project, decision, tradeoff, or failure mode. Answers are grounded in the Build Ledger.

3) **Validate in the Build Ledger**
Each project is documented as: system design, constraints, decisions, failure log, metrics, and next proof.
```

Projects

GetIT — Coordination Layer for Services
A video-first coordination layer that converts trust into booked outcomes—designed to become ambient-native infrastructure.

- Density-first growth (micro-area clusters)
- Trust → booking → payment
- Context layer + modular agents

Best entry point: Build Ledger → GetIT

Fast Food — Decision Compression Engine for Meals
An operating system that collapses “what’s for dinner?” into a sub-3-minute decision for exhausted households.

- Local-first decisioning + selective cloud sync
- Context-aware suggestions (time, weather, preferences)
- Calm, immersive minimal UI (“living canvas” direction)

Best entry point: Build Ledger → Fast Food

Anansi — Knowledge Acquisition OS
An AI-native lesson-to-game system that converts standards into playable, Chromebook-friendly indie-grade quests with measurable mastery reporting.

- Lesson-to-game generator
- Mastery capture + credential-ready reports
- Wedge-first standards unit pilots (TEKS-aligned)

Best entry point: Build Ledger → Anansi

What I optimize for

- Liquidity before scale
- Trust before volume

- Local-first when it improves speed, privacy, and resilience
- Measurable loops before philosophy
- Complexity budgets and reversible architecture

If you want to talk

Use **Fit Check** first.
If the fit is strong, we can move quickly.

---

2) Ask Me Anything System Prompt (Operator voice + citations)

Use this as the server-side system prompt for your chat agent.

You are “Prentiss Operator Interface”: a calm, minimal, evidence-backed assistant that answers questions about Prentiss’s work.

Primary goal:
Help hiring leaders evaluate demonstrated capability through specific, grounded answers linked to evidence.

Hard rules:
- Never invent facts, metrics, or project details.
- If evidence is missing, say: “I don’t have that documented yet,” then suggest exactly what artifact would close the gap.
- Prefer operational clarity: constraints, tradeoffs, risks, metrics, execution steps.
- Keep tone calm, minimal, confident. No hype. No defensiveness.
- Always include citations as links to the relevant Build Ledger section anchors when you reference a claim.
- If asked about future plans, separate “current evidence” from “planned direction.”

Output format (always):
1) Direct answer (2–6 sentences)
2) Evidence (bullets, each with a title + URL/anchor)
3) Risks / tradeoffs (bullets)
4) Suggested follow-ups (2–3 short questions)

If the user asks a vague question:

- Ask one clarifying question max, then give a best-effort answer based on existing evidence.

Implementation note: enforce “Evidence required” at the application layer: if retrieval returns no chunks above threshold, the agent must respond with “not documented yet.”

---

3) JSON Schema for Project Ingestion (future-proof, still simple)

Save as: `lib/schema/project.ts` (Zod) or `schema/project.json`. Here’s the JSON Schema version:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://example.com/schemas/project.json",
  "title": "Project",
  "type": "object",
  "required": [
    "slug",
    "name",
    "oneLiner",
    "systemDesignPlain",
    "twoWeekNextSteps",
    "constraints",
    "decisions",
    "metrics",
    "failureLog"
  ],
  "properties": {
    "slug": { "type": "string", "minLength": 2 },
    "name": { "type": "string", "minLength": 2 },
    "status": { "type": "string", "enum": ["active", "paused", "concept"] },
    "oneLiner": { "type": "string", "minLength": 10 },
    "systemDesignPlain": { "type": "string", "minLength": 20 },
    "twoWeekNextSteps": { "type": "string", "minLength": 20 },

    "constraints": {
      "type": "array",
      "items": { "type": "string", "minLength": 2 },
      "minItems": 3
```

```
},

"decisions": {
  "type": "array",
  "minItems": 3,
  "items": {
    "type": "object",
    "required": ["title", "optionA", "optionB", "chosen", "why", "tradeoff"],
    "properties": {
      "title": { "type": "string", "minLength": 3 },
      "optionA": { "type": "string", "minLength": 3 },
      "optionB": { "type": "string", "minLength": 3 },
      "chosen": { "type": "string", "enum": ["A", "B"] },
      "why": { "type": "string", "minLength": 10 },
      "tradeoff": { "type": "string", "minLength": 10 },
      "reversalTrigger": { "type": "string", "minLength": 10 }
    },
    "additionalProperties": false
  }
},

"metrics": {
  "type": "array",
  "items": {
    "type": "object",
    "required": ["name", "definition"],
    "properties": {
      "name": { "type": "string", "minLength": 3 },
      "definition": { "type": "string", "minLength": 10 },
      "target": { "type": "string" },
      "timeWindow": { "type": "string" }
    },
    "additionalProperties": false
  },
  "minItems": 3
},

"failureLog": {
  "type": "object",
  "required": ["failure", "whatBroke", "lesson", "permanentChange"],
  "properties": {
    "failure": { "type": "string", "minLength": 10 },
    "whatBroke": { "type": "array", "items": { "type": "string", "minLength": 5 }, "minItems": 1 },
    "lesson": { "type": "string", "minLength": 10 },
```

```
      "permanentChange": { "type": "string", "minLength": 10 }
    },
    "additionalProperties": false
  },

  "artifacts": {
    "type": "array",
    "items": {
      "type": "object",
      "required": ["label", "url"],
      "properties": {
        "label": { "type": "string", "minLength": 2 },
        "url": { "type": "string", "minLength": 5 },
        "kind": { "type": "string", "enum": ["doc", "repo", "demo", "video", "design", "other"] }
      },
      "additionalProperties": false
    }
  }
},
"additionalProperties": false
}
```

80/20 improvement baked in: `reversalTrigger` is included. That single field increases credibility a lot.

---

4) Next.js App Router Folder Structure (clean + staged for RAG + WebGL)

This assumes: Next.js App Router + MDX content.

```
/app
  /layout.tsx
  /page.tsx                    (redirect to /start-here or render start-here)
  /start-here/page.tsx
  /build-ledger/page.tsx          (index)
  /build-ledger/[slug]/page.tsx   (mdx project pages)
  /notes/page.tsx                (index)
  /notes/[slug]/page.tsx         (mdx notes)
  /fit-check/page.tsx
```

```
  /ask/page.tsx
  /api
    /ask/route.ts              (chat endpoint: retrieval + generation)
    /fit-check/route.ts         (optional: server-side eval + storage)
    /content-index/route.ts      (optional: rebuild embeddings)
/components
  /Shell.tsx
  /Nav.tsx
  /Card.tsx
  /Prose.tsx                   (typography wrapper)
  /DecisionCard.tsx
  /MetricList.tsx
  /FailureLog.tsx
  /FitCheckForm.tsx
  /FitCheckResult.tsx
  /Chat.tsx
  /Citations.tsx
  /Motion.tsx                  (minimal motion helpers + reduce-motion)
/content
  /projects
    getit.mdx
    fast-food.mdx
    anansi.mdx
  /pages
    start-here.mdx
  /notes
    (future)
/lib
  /content
    mdx.ts                     (MDX loader)
    index.ts                   (content registry)
  /schema
    project.ts                 (Zod mirror of schema)
  /rag
    chunk.ts                   (mdx -> chunks)
    embed.ts                   (embedding client)
    retrieve.ts                (top-k retrieval)
    cite.ts                    (citation formatting)
  /fit
    evaluate.ts                (prompt + structured output parsing)
  /utils
    urls.ts                    (anchor building)
    text.ts
/public
```

```
  /og
  /fonts
/styles
  globals.css
```

Where WebGL will live (later, staged)

- `/components/canvas/AmbientCanvas.tsx`
- loaded only on the homepage via dynamic import
- respects `prefers-reduced-motion`
- never blocks LCP

---

5) The “Auto-prompt System Design + 2-week proof” requirement (locked in)

Add this to onboarding spec (for the future platform):

When a user uploads + certifies a project, the system must automatically generate:

- `systemDesignPlain`
- `twoWeekNextSteps`

without asking.

Only ask the user to confirm/edit:

- “Is this accurate?” ✅ / ✏️

If confidence is low (missing evidence), show:

- “I can’t infer the system design from the uploaded artifacts. Upload one of: architecture doc, README, roadmap, or demo transcript.”

That keeps it frictionless **and** honest.

MDX Files

Below are **ready-to-paste MDX files** for the 3 projects **plus** the two index pages:

- `content/projects/getit.mdx`
- `content/projects/fast-food.mdx`
- `content/projects/anansi.mdx`
- `content/pages/build-ledger-index.mdx` (used by `/build-ledger`)
- `content/pages/notes-index.mdx` (used by `/notes`)

They include **stable section anchors** (for citations), **Decision Cards**, **Reversal Triggers**, and the **2-week proof** section.

---

1) `content/projects/getit.mdx`

```
---
slug: "getit"
title: "GetIT"
subtitle: "Coordination Layer for Services"
status: "active"
oneLiner: "A video-first coordination layer that converts trust into booked outcomes—designed to become ambient-native infrastructure."
description: "System design, constraints, decisions, failure log, metrics, and the next density proof."
tags: ["marketplace", "coordination", "local-services", "agents", "ambient"]
---

# GetIT
## Coordination Layer for Services

<p className="text-sm opacity-70">
Last updated: 2026-03-04
</p>

---

## One-liner {#one-liner}

A video-first coordination layer that converts trust into booked outcomes—designed to become ambient-native infrastructure.

---
```

System design (plain English) {#system-design}

I run a video-first local services feed that converts trust into transactions, powered by a context layer (time, location, behavior) and modular agents that route discovery → intent → booking → payment.

Constraints {#constraints}

- Liquidity per square mile
- Trust formation speed
- Vendor density and supply quality
- Fraud and payment disputes
- Intent → booking friction
- Ambient-native compatibility (voice + screenless discovery paths)

Decision cards {#decisions}

1) Marketplace vs coordination layer {#decision-marketplace-vs-coordination}

A: Build another local services marketplace
B: Build the invisible coordination layer for services

I chose: **B**

I reframed GetIT from “TikTok for Business” into a transactional coordination layer.
Not listings. Not directories. Not social.

Why: Listings commoditize. Coordination compounds.

Tradeoff: This increases systems complexity before the market loop is proven.

Reversal trigger: If density cannot be proven in a micro-area with a simplified funnel, I narrow scope to a single vertical workflow until bookings compound.

---

2) Ads vs trust-driven density loops {#decision-ads-vs-density}

A: Paid acquisition + broad geo expansion
B: High-density micro-area clusters + early partner flywheel

I chose: **B**

Houston Heights wedge. Local pro clusters. Invite-only launch.

Why: Liquidity before scale. Trust before volume. Signal before noise.

Tradeoff: Slower early growth; requires disciplined local execution.

Reversal trigger: If micro-area density fails after multiple funnel simplifications, I pause expansion and rebuild vendor quality + category focus before adding supply.

---

3) Screen app vs ambient OS {#decision-screen-vs-ambient}

A: Stay a mobile-first booking app
B: Design toward ambient-native agent infrastructure

I chose: **B**

I made a deliberate bet on:
- Voice intent parsing
- Context layer + modular agents
- SDK path for screenless discovery

Why: Avoid future rebuild. Position for where interfaces are going.

Tradeoff: Adds roadmap gravity and engineering ambition.

Reversal trigger: If near-term bookings require extreme simplicity, ambient work becomes a parallel track only after density/retention are validated.

Failure log {#failure-log}

Failure: Engineering sophistication outpaced market validation.

I built advanced layers (editor, tagging, personalization, modularization) before proving repeat booking velocity in a single dense vertical.

What broke:
- Signal hierarchy (complex ML before demand compounding)

- Energy dilution (multiple advanced layers without ruthless simplification)

Lesson: Liquidity > intelligence early.

Permanent change: All future roadmap work gates behind density + repeat booking proof in a micro-area.

Metrics that matter {#metrics}

1) **Booking density per square mile**
If you don't win density, you don't win.

2) **Repeat booking rate (30 / 60 / 90 day)**
Trust platforms win on habit, not downloads.

3) **Time-to-intent fulfillment**
From "I need X" → confirmed booking. This becomes the ambient-native north star.

2-week next proof {#two-week-proof}

Run a density proof in one Houston micro-area:

- **15–25 vendors**
- **2–3 categories**
- Measure **booking velocity** and **repeat behavior**

If bookings don't compound locally, I simplify the funnel until they do.

What I would do next (after the proof) {#next-after-proof}

If density compounding appears:
- double down on the micro-area cluster playbook
- standardize vendor onboarding + quality controls
- tighten the conversion path to reduce time-to-intent

If density compounding does not appear:
- reduce categories to one wedge
- simplify steps until the first loop is undeniable

- pause intelligence layers until the loop holds

Artifacts (optional) {#artifacts}

Add links here as they become publishable: demos, docs, repos, roadmaps.

---

2) `content/projects/fast-food.mdx`

```
---
slug: "fast-food"
title: "Fast Food"
subtitle: "Decision Compression Engine for Meals"
status: "active"
oneLiner: "An operating system that collapses “what’s for dinner?” into a sub-3-minute decision for exhausted households."
description: "System design, constraints, decisions, failure log, metrics, and the next household loop proof."
tags: ["local-first", "decisioning", "consumer", "ux", "context"]
---

# Fast Food
## Decision Compression Engine for Meals

<p className="text-sm opacity-70">
Last updated: 2026-03-04
</p>

---

## One-liner {#one-liner}

An operating system that collapses “what’s for dinner?” into a sub-3-minute decision for exhausted households.

---

## System design (plain English) {#system-design}
```

I collect lightweight context signals (time, weather, preferences, household patterns) and use a local-first decision engine to compress dinner decisions into one clear, fast recommendation with calm, immersive UX.

Constraints {#constraints}

- Cognitive fatigue (minimal tolerance for friction)
- Habit formation (repeat use matters more than feature breadth)
- Privacy expectations (avoid creepiness)
- Offline resilience (must function under poor connectivity)
- Context accuracy (avoid “smart” suggestions that feel wrong)
- Minimal UI tolerance (reduce decision width)

Decision cards {#decisions}

1) Recipe app vs decision compression engine {#decision-recipe-vs-compression}

A: Make a beautiful meal planning app
B: Build a cognitive load reducer for exhausted humans

I chose: **B**

This isn’t culinary content. It’s mental relief.

Why: Exhausted users don’t want browsing. They want resolution.

Tradeoff: Requires restraint; fewer features can look “too simple” unless outcomes are strong.

Reversal trigger: If users consistently want exploration, I add a secondary browse mode—but keep the primary path as “one answer.”

---

2) Cloud AI vs local-first intelligence {#decision-cloud-vs-local}

A: Fully cloud dependent
B: Local-first with Supabase sync

I chose: **B**

- Local-first decisioning
- Weather integration
- Context-aware suggestions
- Supabase long-term learning

Why: Speed, privacy, resilience.

Tradeoff: Harder personalization constraints than “just call the cloud.”

Reversal trigger: If local constraints prevent meaningful personalization, shift heavier reasoning to cloud while keeping offline fallback intact.

---

3) Feature expansion vs living canvas philosophy {#decision-features-vs-canvas}

A: Add more features
B: Build a calm, immersive, ambient UI system

I chose: **B**

Experience identity > feature count.

Why: The product promise is calm clarity, not feature abundance.

Tradeoff: Requires taste + discipline; visual work must serve behavior, not aesthetics.

Reversal trigger: If calm minimal UI reduces clarity, I prioritize comprehension over ambiance and simplify the visual system.

Failure log {#failure-log}

Failure: Philosophical clarity outpaced user volume testing.

The “why” is clear, but the daily loop isn’t proven yet.

What broke:
- No measured “exhausted household” retention loop yet
- No proof that decision compression translates into habit

Lesson: Validate repetition before refining transcendence.

Permanent change: All UI elevation gates behind demonstrated repeat usage and acceptance rate.

Metrics that matter {#metrics}

1) **Time to dinner decision** (target: under 3 minutes)
2) **Weekly active household rate**
3) **Decision acceptance rate** (did they cook what was suggested?)

2-week next proof {#two-week-proof}

Test a 14-day exhausted-household loop:

- **20 families**
- Measure:
  - decision time (<3 min)
  - acceptance rate
  - repeat usage

If overrides are high, tighten logic and reduce choice width.

What I would do next (after the proof) {#next-after-proof}

If the loop sticks:
- formalize the “3-minute path”
- add lightweight learning that improves without adding choice
- scale to more households with minimal onboarding friction

If the loop fails:
- reduce inputs (less context, fewer options)
- tighten recommendation confidence
- identify the minimum viable decision engine that gets repeated use

---

Artifacts (optional) {#artifacts}

Add links here as they become publishable: prototype videos, UX specs, architecture notes.

---

3) `content/projects/anansi.mdx`

```
---
slug: "anansi"
title: "Anansi"
subtitle: "Knowledge Acquisition OS"
status: "active"
oneLiner: "An AI-native lesson-to-game system that converts standards into playable, Chromebook-friendly indie-grade quests with measurable mastery reporting."
description: "System design, constraints, decisions, failure log, metrics, and the next standards-unit pilot."
tags: ["education", "games", "standards", "mastery", "chromebook"]
---

# Anansi
## Knowledge Acquisition OS

<p className="text-sm opacity-70">
Last updated: 2026-03-04
</p>

---

## One-liner {#one-liner}

An AI-native lesson-to-game system that converts standards into playable, Chromebook-friendly indie-grade quests with measurable mastery reporting.

---

## System design (plain English) {#system-design}

I convert standards-aligned lessons into playable, Chromebook-friendly indie-grade quests using an AI lesson-to-game generator, then capture mastery data and output credential-ready reports for teachers and parents.

---

## Constraints {#constraints}
```

- Standards alignment (e.g., TEKS)
- Chromebook constraints (performance + browser)
- Classroom time budgets
- Engagement vs rigor balance
- Teacher reporting requirements
- Proof of learning (not just playtime)

Decision cards {#decisions}

1) Content platform vs knowledge OS {#decision-content-vs-os}

A: Educational content platform
B: Knowledge acquisition operating system

I chose: **B**

This is an OS: generation → play → mastery capture → reporting.

Why: Content is abundant. Measurable learning loops are scarce.

Tradeoff: Harder product to ship; requires correct measurement and teacher-grade outputs.

Reversal trigger: If OS scope slows adoption, ship a single “quest generator + report” wedge first and expand only after proven lift.

---

2) Gamified app vs judgment training engine {#decision-fun-vs-judgment}

A: Make learning fun
B: Train long-term decision-making capacity

I chose: **B**

Games are the delivery mechanism. Cognitive durability is the value.

Why: Engagement without learning lift is empty.

Tradeoff: Harder to measure and communicate; requires credible assessments.

Reversal trigger: If rigor harms engagement, adjust pacing and feedback loops—without lowering standards alignment.

---

3) Broad platform vs narrow standards wedge {#decision-broad-vs-wedge}

A: Broad multi-subject platform
B: One narrow standards unit pilot

I chose: **B**

One unit. One cohort. One measurement loop.

Why: OS-level ideas need concrete entry portals.

Tradeoff: Slower expansion; requires discipline.

Reversal trigger: If the pilot proves mastery lift + replay, expand to adjacent units before expanding subjects.

Failure log {#failure-log}

Failure: Early abstraction level was too high for implementation.

The concept was strong, but the entry wedge was initially fuzzy.

Lesson: Every OS needs a painfully concrete portal.

Permanent change: Anansi ships as standards-unit pilots first. Expansion happens only after measured mastery lift.

Metrics that matter {#metrics}

1) **Completion rate**
2) **Replay rate**
3) **Pre/post mastery lift**

2-week next proof {#two-week-proof}

Pilot one narrow standards unit (e.g., TEKS-aligned math or science):

- Generate **5–10 playable quests**
- Test with **10–15 students**
- Measure:
  - completion
  - replay
  - pre/post mastery lift

If rigor or engagement breaks, fix the generator before expanding scope.

What I would do next (after the proof) {#next-after-proof}

If lift is real:
- expand to adjacent units inside the same subject
- improve reporting clarity for parents/teachers
- formalize generator constraints to keep quality consistent

If lift is weak:
- tighten the standards mapping
- adjust quest pacing and feedback
- improve mastery measurement before adding content volume

Artifacts (optional) {#artifacts}

Add links here as they become publishable: quest samples, pilot protocol, reporting mockups.

---

4) Build Ledger Index Page (MDX)

Save as: `content/pages/build-ledger-index.mdx`

---
title: "Build Ledger"
description: "Projects documented as decisions, constraints, failures, metrics, and next proofs."

Build Ledger

A calm, evidence-backed record of what I’m building and why.

Each project is documented with:
- system design (plain English)
- constraints
- decision tradeoffs (including reversal triggers)
- failure log
- metrics
- next proof (2-week test)

Projects

- **GetIT** — Coordination layer for services
  Density-first trust → booking → payment, designed toward ambient-native infrastructure.

- **Fast Food** — Decision compression engine for meals
  Local-first recommendations that reduce cognitive load for exhausted households.

- **Anansi** — Knowledge acquisition OS
  Standards-aligned lesson-to-game generation with mastery capture and reporting.

---

5) Notes Index Page (MDX)

Save as: `content/pages/notes-index.mdx`

---
title: "Operating Notes"
description: "Short, operational notes on frontier tech—constraints, tradeoffs, and proofs."

Operating Notes

Short notes on frontier systems work: agents, ambient UX, on-device intelligence, trust, and measurement.

Rule: each note includes at least one of:
- a constraint
- a tradeoff

- a metric
- a failure mode
- a working proof

No notes published yet.

Optional (but helpful): MDX rendering note

If you want the anchors like `{#system-design}` to work reliably, ensure your MDX pipeline uses a heading slug plugin OR keep explicit anchors as shown.

UI

familiar entry → immersive exploration

Users should **start with something recognizable**, then **fall into a discovery space** that feels new.

Think:

- Google → search box → knowledge graph
- Figma → canvas → infinite exploration
- Apple Maps → zoom levels → different modes

North Star needs the same **progressive reveal**.

---

1. The Core Metaphor

North Star should feel like **exploring a living constellation**.

Not a feed.
Not a directory.

A **map of capability**.

Nodes represent:

- **People**
- **Projects**
- **Decisions**
- **Patterns**

Connections represent:

- shared decisions
- shared constraints
- collaboration
- influence

Visually:

```
Decision
   *
```

```
      / \
     /   \
  Person   Project
     \     /
      \   /
     Pattern
```

But users **never see this as a raw graph**.

It's rendered as **a calm constellation field**.

---

2. The Entry Experience

When someone lands on North Star they see something familiar.

Mode 1 — The Command Bar

Center screen:

Explore capability

[Search people, projects, or problems]

Examples under it:

operators who built density-first marketplaces
people solving AI governance
edtech systems with measurable learning loops

This is **LLM familiarity**.

No learning curve.

After Search → The Constellation

Instead of a list, the results **materialize as a field of nodes**.

Subtle animation.

Like stars appearing.

Users see:

- clusters
- proximity
- patterns

This is the **Capability Matrix**.

---

3. The Capability Matrix Interface

Think **Apple Maps meets a neural network**.

The canvas

Dark, calm background.

Soft gradient.

Nodes gently glowing.

No noise.

Example layout:

```
        Person
          *
         / \
        /   \
    Project   Decision
      *         *
       \       /
        \     /
         Pattern
```

Clusters emerge organically.

Users immediately see:

- who is close to whom
- which ideas connect
- where expertise lives

---

4. Interaction Model

This is where immersion happens.

Hover

Hovering over a node reveals a **halo**.

Example:

Prentiss
Frontier Operator

Projects:
GetIT
Fast Food
Anansi

Lightweight.

Click → Expand

Clicking a node expands it.

Example:

GETIT
Coordination Layer for Services

Decisions:
• Density-first marketplaces
• Coordination vs listing
• Ambient architecture

Metrics:

• Booking density
• Repeat bookings

Connections animate outward.

You see **how this node relates to others**.

Zoom

Zooming out reveals **pattern clusters**.

Example clusters:

Marketplaces
Local-first AI
Education systems
Agent architectures

Zooming in reveals **specific people and projects**.

Exactly like maps.

---

5. Multiple Exploration Modes

Users should be able to **switch perspectives**.

A small toggle in the corner.

Mode A — People

Nodes represent **people**.

Edges represent:

- collaboration
- shared decisions

- shared project patterns

Hiring leaders love this.

Mode B — Projects

Nodes represent **projects**.

Edges represent:

- shared architecture
- shared market type
- shared metrics

Founders love this.

Mode C — Decisions

Nodes represent **decisions**.

Example:

local-first architecture
density-first marketplace
agent modularization

Edges show **who chose them**.

This becomes incredibly powerful.

Mode D — Patterns

Nodes represent **extracted patterns**.

Example:

Infrastructure builders
Local-first thinkers

Agent-first architects
Density-first marketplaces

Users see **clusters of people with similar thinking.**

---

6. The Profile View (Entering a Node)

Clicking a person node opens their **Operator Surface**.

Right side panel slides out.

Layout:

Prentiss

Frontier Operator

Patterns detected:
Infrastructure-first
Long-horizon architecture
Density marketplaces
Local-first AI

Projects
• GetIT
• Fast Food
• Anansi

Ask
[ask about this operator]

The constellation remains visible behind the panel.

So you never lose **context**.

---

7. The “Ask the Graph” Feature

Instead of asking a person, you can ask the **network**.

Example query:

How do builders solve marketplace cold start?

The graph highlights nodes:

- projects where it was solved
- decisions that enabled it
- operators who did it

Visually:

Nodes glow.

Edges pulse.

This is **beautiful and useful**.

---

8. The Emotional Experience

North Star should feel:

- calm
- intelligent
- expansive
- slightly magical

Not:

- noisy
- corporate
- gamified
- cluttered

Users should feel:

> “I’m exploring knowledge.”

Not:

> “I’m scrolling a platform.”

---

9. Why This Works Psychologically

Humans understand knowledge best when it’s:

- **spatial**
- **relational**
- **navigable**

Text feeds hide relationships.

Maps reveal them instantly.

The capability matrix makes **expertise visible.**

---

10. The 80/20 MVP Version

You don’t need insane complexity at first.

Start with:

Nodes:

people
projects

Edges:

built
worked-on

Then gradually add:

decisions
patterns
constraints

metrics

The system becomes richer over time.

---

11. Technology Approach (Practical)

You can build this with:

- **WebGL**
- **Three.js**
- **React Three Fiber**
- **Graph data structure**

But keep it **simple first**.

Even a basic force-directed graph with elegant motion can feel powerful.

---

12. The Long-Term Vision

North Star eventually becomes:

A map of human capability

You explore:

- problems
- operators
- systems
- decisions

All connected.

It’s a **living atlas of builders**.

---

13. Why This Could Be Huge

Because the world increasingly needs:

- **operators**
- **systems thinkers**
- **real builders**

North Star makes them **discoverable**.

Not by titles.

But by **evidence**.

You're essentially building a **cosmic-scale knowledge graph renderer**.
Not just nodes + edges — a **living constellation system** that reveals structure as you zoom out.

Below is a **developer-ready breakdown of the three visual layers** your images represent and how engineers should implement the beauty layer on top of the graph math.

Immersive Knowledge Graph Visual System

(Developer Breakdown)

Core Concept

The visualization represents **knowledge as a universe**.

Every node = an individual profile, skill, or capability.
Edges = relationships, similarity, collaboration, or shared attributes.

As the camera zooms out:

1. **Single node → connections**

2. **Cluster → community**
3. **Meta-cluster → constellations**
4. **Entire network → universe**

Each zoom level changes **visual representation, density, and abstraction**.

Layer 1 — Local Graph (Image 1)

What the user sees

A **dense graph of nodes and edges with data flowing between them**.

This is the **microscopic view**.

Visual Components

Nodes

Representation

circle
radius: 6–10px
glow radius: 12–20px

Color encoding example

cyan = verified capability
magenta = trending node
white = neutral

Node glow

outer blur: gaussian
opacity: 0.6

Edges

Edges are **curved splines**, not straight lines.

Use:

cubic bezier
or
catmull-rom spline

Edge properties

width: 1–2px
opacity: 0.4

Edge color gradient

start = node A color
end = node B color

Flow Animation

Edges should have **data pulse motion**.

Implementation

animated dash offset
or
particle travel along path

Example

pulse speed: 1.2s
particle size: 2px
particle glow: 4px

Numeric Labels

Small floating metadata values near edges.

font size: 10px
opacity: 0.6
color: same as edge

Fade these out at medium zoom.

Layout Engine

Use **force-directed layout**.

Recommended engines

d3-force
or
forceAtlas2
or
ngraph

For GPU scale:

Sigma.js
or
Graphology
or
Three.js + custom layout

Layer 2 — Cluster Field (Image 2)

What the user sees

Individual nodes begin to **merge visually into clusters**.

Edges become **geometric meshes**.

The network begins to look like **organic constellations**.

Visual Components

Cluster Hulls

Clusters should have a **soft glow boundary**.

Technique

alpha shape
or
convex hull

Render as

semi-transparent gradient blob
opacity: 0.15
blur: 30px

Color

derived from cluster average node color

Edge Mesh

Edges form **triangular mesh shapes**.

Technique

Delaunay triangulation

Render

thin glowing triangles
opacity: 0.25

Purpose

communicates group cohesion

Node Aggregation

Cluster node display changes.

Instead of many nodes:

cluster node
radius: 20–40px

Cluster pulse animation

slow breathing glow
duration: 4s
scale: 1.05

Background Atmosphere

Add **color fog layers** behind clusters.

Implementation

radial gradient textures
blur: 80–120px
opacity: 0.15

This creates the **nebula effect** seen in the reference.

Layer 3 — Universe View (Image 3)

What the user sees

The entire network becomes **cosmic fiber flows**.

Clusters become **galaxies**.

Edges become **vector fields**.

Visual Components

Cluster Galaxies

Clusters render as **particle clouds**.

Implementation

10k–100k particles

Particle attributes

size: 1–3px
opacity: 0.6
motion: slow drift

Inter-Cluster Streams

Large connections become **energy streams**.

Technique

noise-driven splines

Add turbulence

perlin noise

Animation

flow speed: 4–8s loop
width: 4–10px
glow: strong

Motion Field

Background particles should **move subtly**.

Use

velocity field

or

noise field drift

Purpose

Creates the **living universe feeling**.

Rendering Architecture

To achieve this scale **you must use GPU rendering**.

Recommended stack

WebGL
Three.js
or
PixiJS

For very large graphs:

deck.gl
regl

Performance Requirements

Target

60fps

Graph size tiers

Small

1k nodes
5k edges

Medium

10k nodes
50k edges

Large

100k nodes
500k edges

For large graphs:

use spatial indexing
quad tree
LOD rendering

Zoom System

Zoom controls what renders.

Zoom Level 1

scale < 1.5

Render

galaxies
cluster clouds
streams

Hide

nodes
labels
edges

Zoom Level 2

1.5–4

Render

cluster hulls
mesh edges
major nodes

Zoom Level 3

> 4

Render

individual nodes
edges
labels
data pulses

Color System

Use **additive blending**.

blend mode: screen

Palette

cyan
magenta
violet
yellow
electric blue

Nodes should **glow like stars**.

Interaction Model

Hover node

Show

profile card
skills
connections

Animation

node expands
radius +4px
glow increases

Click node

Camera transition

ease duration: 900ms
curve: cubic

Zoom into local network.

Motion Principles

Animations must feel **cosmic and fluid**.

Avoid

linear movement

Use

easing curves
noise drift
particle flow

Visual Polish Checklist

Developers should implement

✓ node glow shaders
✓ edge gradients
✓ pulse particles
✓ cluster hulls
✓ particle atmospheres
✓ noise drift
✓ additive blending
✓ zoom-based LOD rendering

If Done Correctly

Users should feel like they are:

Flying through a living map of human capability.

Zoom in:

people
skills

relationships

Zoom out:

communities
industries
civilization

Capability Matrix

Here's an elegant, robust data model that gives you the **capability matrix** without turning into a swamp. It's designed for:

- fast search + filtering
- graph exploration (people/projects/decisions/patterns)
- RAG citations into specific ledger sections
- scalable onboarding (upload → extract → confirm → publish)
- future "interconnected living matrix" growth

I'm going to give you:

1. the core entities (minimal set)
2. relationships (graph edges)
3. the derived layer (patterns + embeddings)
4. permissions + trust model (so it doesn't become LinkedIn spam)
5. a Postgres/Supabase-friendly schema outline

---

1) First Principles

Keep the core graph small

Don't model everything as a node. Model only what you need to explore.

Core nodes:

- Person
- Project
- Claim (decision / constraint / metric / failure / next proof)
- Artifact (evidence)
- Query (fit check + ask-the-graph results)

Everything else becomes either:

- properties
- derived tags
- embeddings

Evidence is first-class

LinkedIn dies because it's claims without verifiability.

North Star must be:

> claim → evidence → interrogation

So every Claim must be linkable to:

- Artifact(s)
- Ledger anchor(s)

---

2) Core Entity Model (Elegant)

A) `person`

Represents an operator.

Key fields:

- `id`
- `handle` / `slug`
- `name`
- `headline`
- `summary`
- `location` (optional)
- `visibility` (public / unlisted / private)
- `created_at`, `updated_at`

B) `project`

A structured build ledger node.

Key fields:

- `id`
- `person_id` (owner)
- `slug`
- `title`

- `subtitle`
- `status` (active / paused / concept)
- `one_liner`
- `system_design_plain`
- `two_week_proof`
- `published_at`
- `updated_at`

C) `claim`

This is the heart of the model.

A “claim” is any structured piece of the ledger that can be connected across profiles.

Types:

- `decision`
- `constraint`
- `metric`
- `failure`
- `next_proof`
- `pattern` (derived later)

Key fields:

- `id`
- `project_id`
- `type`
- `title` (short)
- `body` (long)
- `meta` (JSONB) — structured per type
- `confidence` (system extraction confidence)
- `created_at`

Examples of `meta`:

Decision meta

```
{
  "optionA": "Build another marketplace",
```

```
  "optionB": "Build coordination layer",
  "chosen": "B",
  "tradeoff": "Complexity before validation",
  "reversal_trigger": "If density doesn't compound..."
}
```

Metric meta

```
{
  "definition": "Bookings per sq mile",
  "target": null,
  "window": "30d"
}
```

Failure meta

```
{
  "what_broke": ["Signal hierarchy", "Energy dilution"],
  "lesson": "Liquidity > intelligence early",
  "permanent_change": "Gate roadmap behind density proof"
}
```

This is elegant because:

- all ledger components become uniform “claims”
- claims become graph nodes later
- onboarding can extract into one consistent format

D) `artifact`

Evidence objects.

Key fields:

- `id`
- `person_id`
- `project_id` (nullable)
- `kind` (doc / repo / demo / video / design / other)
- `title`

- `url` or storage reference
- `hash` (optional for integrity)
- `visibility`
- `created_at`

E) `citation`

Links a claim to evidence.

Key fields:

- `id`
- `claim_id`
- `artifact_id` (nullable)
- `source_type` (mdx / doc / transcript / url)
- `source_ref` (e.g., `content/projects/getit.mdx`)
- `anchor` (e.g., `#decision-ads-vs-density`)
- `quote` (optional, short)
- `created_at`

This is what makes “Ask” answers credible.

---

3) Relationship Model (The Graph Edges)

You need a generic edge table so the matrix can grow without schema thrash.

`edge`

Represents a relationship between nodes.

Key fields:

- `id`
- `from_type` (person / project / claim)
- `from_id`
- `to_type` (person / project / claim)

- `to_id`
- `edge_type` (built / contains / supports / relates / similar_to / derived_from)
- `weight` (float)
- `meta` (JSONB)
- `created_at`

Common edges

- person → project: `built`
- project → claim: `contains`
- claim → artifact: `supports`
- claim ↔ claim: `similar_to` (semantic)
- person ↔ person: `collaborated_with` (optional later)

This lets the constellation render:

- People
- Projects
- Decisions/metrics/failures
- Connections

without requiring 50 join tables.

---

4) Derived Intelligence Layer (Keeps it "Living")

This is what powers:

- pattern clusters
- similarity edges
- ask-the-graph answers
- capability search

A) `embedding`

Store vectors for semantic search.

Key fields:

- `id`
- `node_type` (person / project / claim / artifact)
- `node_id`
- `embedding` (vector)
- `model`
- `created_at`

You'll primarily embed:

- claims
- project summaries
- artifacts (optional)
- notes posts

B) `pattern`

Patterns are *derived*, not user-written.

A pattern is a normalized label like:

- "density-first marketplaces"
- "local-first systems"
- "agent modularization"

Key fields:

- `id`
- `label`
- `description` (optional)
- `created_at`

C) `pattern_membership`

Many-to-many:

- pattern ↔ claim
- pattern ↔ project
- pattern ↔ person

Key fields:

- `pattern_id`
- `node_type`
- `node_id`
- `strength` (float)
- `created_at`

This is how you get “Pattern Mode” in the graph.

---

5) Interaction Model Tables (So the product works)

A) `fit_check`

Stores evaluations.

Key fields:

- `id`
- `requester_id` (nullable if public)
- `person_id` (whose profile evaluated)
- `input_text`
- `result_json`
- `created_at`

B) `graph_query`

Ask-the-graph interactions.

Key fields:

- `id`
- `requester_id` (nullable)
- `query_text`
- `result_nodes` (JSONB)
- `result_edges` (JSONB)
- `created_at`

These become:

- shareable links
- viral loop (“here’s the graph for your problem”)

---

6) Robust Trust + Anti-Spam Model (Critical)

North Star dies if people can fake it.

So add two score mechanisms:

A) `signal_score` (computed)

Per person/project:

- evidence density
- citation coverage
- external artifacts (repos/demos)
- consistency across claims
- peer endorsements (later)

B) `verification_flags`

Optional early system:

- “Has artifacts”
- “Has published failure log”
- “Has measurable metrics”
- “Has 2-week proof”

These are not badges for ego.
They’re cues for hiring leaders.

---

7) Supabase/Postgres Schema Outline (Practical)

Tables:

- `person`
- `project`
- `claim`
- `artifact`
- `citation`
- `edge`
- `pattern`
- `pattern_membership`
- `embedding`
- `fit_check`
- `graph_query`

Indexes you will want:

- `project(person_id, slug)`
- `claim(project_id, type)`
- `edge(from_type, from_id)`
- `edge(to_type, to_id)`
- `pattern(label)`
- vector index on `embedding.embedding`

---

8) Why This Is Elegant

- **One universal unit**: Claim
- **One universal relationship**: Edge
- **Evidence is first-class**: Citation
- **Patterns are derived**: Pattern membership
- **Search works**: Embeddings
- **Graph works**: Edge table
- **Onboarding works**: extraction to claims + citations

It scales cleanly from:

- one person (you)
  to
- 100k profiles

without rewriting your data model.

---

9) The “Profile Creation from Upload” Mapping

When a user uploads:

1. system creates artifacts
2. extracts draft projects
3. extracts claims per project
4. generates citations to anchors or doc chunks
5. asks user to confirm:
    - 3 decisions
    - 1 failure
    - 3 metrics
    - 2-week proof
6. publishes

All of that maps directly to this model.

Visual Language

The interface must communicate three ideas simultaneously:

1. **Individuals are stars**
2. **Ideas create gravity**
3. **Capability forms galaxies**

It should feel less like a website and more like **entering a living universe of builders**.

Below is the **visual language system** that turns that idea into a coherent interface.

The North Star Interface

Visual Language for Human Capability

The interface evolves through **four visual stages** as the network grows.

Stage progression mirrors the growth of the system.

Seed
Constellation
Cluster
Cosmos

The user experiences the world evolving in real time.

Stage 1 — Seed

(The first node)

The system begins with **one living node**.

Think of it as the first star igniting.

Visual behavior:

• subtle pulsing glow
• slow ambient motion
• faint invisible gravitational field

Environment:

Dark canvas.
Very quiet.

*

Hover reveals:

Prentiss
Frontier Operator

Projects
GetIT
Fast Food
Anansi

The node emits **three orbiting satellites** (projects).

This creates immediate structure.

Fast Food

*

GetIT Prentiss Anansi

Edges appear as **soft light filaments**, not lines.

Stage 2 — Constellation

(Projects and decisions appear)

As the profile expands, more nodes appear.

Nodes represent:

Person
Project
Decision
Metric
Failure

Each type has its own **geometry and color language**.

Person node

Shape: sphere
Color: electric cyan
Meaning: operator

Project node

Shape: rounded cube
Color: white

Decision node

Shape: diamond
Color: magenta

Metric node

Shape: ring
Color: amber

Failure node

Shape: fractured shard
Color: red-orange

Connections are **flowing curves**.

Inspired by the first reference image.

The graph now looks like a **constellation map**.

Stage 3 — Cluster

(Multiple people join)

Once multiple operators exist, clusters begin forming.

This stage resembles the second reference image.

Clusters appear because:

• similar decisions
• shared domains
• related technologies

Edges become **triangulated meshes**.

```
Person A
  \   \
   \   Person B
    \ /
  Project X
```

These triangular patterns visually communicate:

collaboration
shared ideas
reinforcing thinking

Clusters glow with **shared color gradients**.

Example cluster:

Marketplace builders

Color signature:

cyan → violet

Another cluster:

AI agent systems

Color signature:

green → yellow

Clusters form **organic geometric meshes**.

The UI feels alive.

Stage 4 — Cosmos

(Large-scale map of capability)

When hundreds or thousands of nodes exist, the map becomes a **capability universe**.

The third reference image is the closest representation.

What emerges visually:

galaxies

Large thematic groups:

AI systems
marketplaces
edtech
biotech
infrastructure

nebulae

Areas of exploration where:

• ideas are emerging
• connections are forming
• structure is still fluid

supernodes

Highly influential operators appear as brighter stars.

Not because of followers.

But because of:

ideas connected
projects built
patterns influencing others

This avoids LinkedIn-style vanity metrics.

Influence emerges naturally from the graph.

Color System

Colors must communicate **capability domains**.

Example palette:

Infrastructure → Cyan
AI systems → Green
Marketplaces → Violet
Education → Yellow
Biotech → Red
Finance → Gold

Clusters blend colors naturally.

This creates the cosmic effect.

Example:

AI + Infrastructure cluster

Becomes:

cyan-green nebula

Motion Language

Motion must communicate **meaning**, not decoration.

Node breathing

Nodes gently pulse every ~6 seconds.

Meaning:

active thinking
living system

Edge activation

When a node is selected, edges brighten and ripple outward.

Meaning:

idea propagation

Cluster drift

Clusters move slowly over time.

Meaning:

evolving knowledge

Movement speed must be extremely slow.

Users should feel calm, not overwhelmed.

Depth Layers

The interface uses **three depth planes**.

foreground

selected nodes
active relationships

midground

neighboring nodes

background

distant clusters

Parallax scrolling creates a subtle **cosmic depth effect**.

Interaction Model

click node

opens right panel with:

operator summary
projects
decisions
metrics
ask interface

drag canvas

navigate space.

scroll

zoom in/out of capability layers.

Zoom Levels

Zoom levels reveal different information densities.

Level 1 — Cosmos

See galaxies.

AI
Marketplaces
Education

Level 2 — Clusters

See groups of operators and projects.

Level 3 — Constellations

See relationships between individuals.

Level 4 — Nodes

See detailed operator profiles.

The Ask Interface

Instead of typing into a static chat box, the user queries the graph.

Example:

How do builders solve marketplace cold start?

The system:

• highlights relevant nodes
• brightens decision edges
• surfaces clusters

Visually it feels like **ideas lighting up in the universe**.

Growth Mechanic

The graph expands organically.

Every time a new operator publishes:

node appears
connections form
cluster adjusts

The world literally grows.

Users witness it happening.

Early Stage Strategy

At launch the map will be sparse.

That’s fine.

You deliberately show **future constellation points**.

Example:

Agent systems
Local-first AI
Marketplace operators
Edtech innovators

Clicking them reveals:

This constellation is forming.
North Star will connect operators working on this idea.

It communicates **inevitable expansion**.

The Emotional Goal

Users should feel three things:

wonder

“I’ve never seen people mapped like this.”

curiosity

“What happens if I zoom here?”

ambition

“I want my node in this universe.”

The Test

The interface succeeds if someone says:

This is LinkedIn if it were built by astrophysicists.

One critical rule

Never allow:

• follower counts
• vanity metrics
• popularity rankings

Influence must emerge only from **ideas and connections**.

That keeps the map intellectually credible.

Be sure to design the **rendering architecture for the constellation engine**:

• node layout algorithms
• clustering math
• WebGL performance strategy
• dynamic edge weighting

That’s the layer that determines whether this vision feels **magical** or **laggy and confusing** once the graph hits thousands of nodes.

Constellation Engine v0 (Single Profile) (CTO)

North Star Constellation Engine v0 (Single Profile)

Document Type: Product + Rendering System Specification
Audience: CTO / Engineering Lead / Graphics Engineer
Version: v0.1
Scope: Minimal Fundable Product — **Single Capability Constellation**

---

1. Purpose

This document defines the **Constellation Rendering Engine v0** for North Star.

The goal of this system is to render a **single human capability constellation** that is:

- visually compelling
- structurally legible
- technically scalable
- performant on commodity laptops

This engine must demonstrate that a person's work, decisions, skills, and outcomes can be represented as a **living constellation graph**.

The system must also subtly imply that this profile is **one node in a much larger universe**.

This document specifies:

- zoom levels and rendering behavior
- node types and visual language
- API contracts
- performance targets
- acceptance criteria for quality

---

2. Product Objective

The constellation must allow a visitor to understand:

- what this person builds
- what decisions shaped their work
- what outcomes resulted
- what capabilities emerge across projects

The system must accomplish this **within 2–3 minutes of exploration**.

The experience must feel closer to:

- a **planetarium**
- a **knowledge atlas**
- a **capability map**

…and **not** like a traditional graph visualization or dashboard.

---

3. Technology Stack

Recommended rendering architecture.

Frontend

- Next.js
- React
- React Three Fiber
- three.js
- @react-three/drei
- Zustand (UI state)

Graph Structure

- graphology
- graphology-layout-forceatlas2
- d3-force (local layout adjustments)

Backend

- Node.js API
- Supabase Postgres

---

4. Scene Architecture

The constellation scene consists of five visual layers.

Layer 1 — Background Field

Purpose: depth and atmosphere

Elements:

- dark gradient sky
- low-density starfield
- subtle noise texture

No semantic meaning.

Layer 2 — Cluster Atmospheres

Purpose: visually define project clusters.

Implementation:

- soft translucent particle clouds
- colored by cluster
- opacity based on cluster gravity

Layer 3 — Edges

Purpose: show relationships between nodes.

Edges render as:

- curved paths
- low alpha
- animated pulse when selected

Edge visibility is limited to reduce visual clutter.

Layer 4 — Nodes

Nodes represent semantic units:

- person
- projects
- decisions
- skills
- outcomes

Nodes are rendered as instanced geometry.

Layer 5 — Interaction Layer

DOM overlay containing:

- node expansion panel
- ask interface
- labels
- tooltips
- navigation UI

---

5. Zoom Levels

The engine supports **four semantic zoom levels**.

Zoom controls visibility, clustering, and label density.

Zoom Level 0 — Universe Hint

Purpose:
Imply that this constellation is part of a larger system.

Visible elements:

- primary profile constellation
- faint ghost nodes in distance
- subtle cluster hints

Ghost nodes are:

- semi-transparent
- non-interactive
- visually distant

Message implied:

> “This is one constellation of many.”

Zoom Level 1 — Profile Constellation (Default)

Default camera view.

Visible elements:

- person node
- project clusters
- major edges

Hidden elements:

- low-importance skill nodes
- weak edges

Goal:

Visitors should immediately see the **structure of the person’s work**.

Zoom Level 2 — Project Cluster

Triggered when:

- clicking a project
- zooming into cluster region

Visible elements:

- project node
- decision nodes
- outcome nodes
- key skills

Edges within cluster become visible.

Goal:

Understand **how a project works internally**.

Zoom Level 3 — Node Focus

Triggered when:

- clicking a node

Behavior:

- camera centers on node
- unrelated nodes dim
- Node Expansion Panel opens

Goal:

Reveal detailed context.

---

6. Node Types

The graph uses **five semantic node types**.

Person Node

Represents the constellation owner.

Visual characteristics:

- brightest node
- warm gold color
- glowing halo
- largest size

Position:

Center of graph.

Purpose:

Anchor of constellation.

Project Node

Represents a major initiative.

Examples:

- GetIT
- Fast Food
- Anansi

Visual characteristics:

- electric blue
- medium-large size
- spherical geometry

Projects become **cluster centers**.

Decision Node

Represents important strategic decisions.

Examples:

- video-first discovery
- density-first market entry
- local-first AI architecture

Visual characteristics:

- violet
- diamond geometry
- slightly smaller than projects

Skill Node

Represents demonstrated capability.

Examples:

- marketplace design
- AI systems
- product architecture

Visual characteristics:

- teal
- small sphere
- minimal glow

Skills should not dominate visually.

Outcome Node

Represents measurable results.

Examples:

- product launch
- vendor onboarding
- pilot deployment

Visual characteristics:

- amber
- square geometry
- moderate glow

Outcomes anchor consequence.

---

7. Edge Types

Edges represent relationships.

Examples:

Relationship	Example
Person → Project	built
Project → Decision	required
Project → Skill	used
Decision → Outcome	produced

Edges use:

- curved geometry
- low opacity
- animated highlight on interaction.

---

8. API Contracts

The engine requires a minimal API surface.

Get Profile Graph

GET /api/graph/profile/{id}

Returns:

```
{
  profile,
  nodes,
```

```
  edges,
  clusters,
  camera
}
```

Nodes include:

- id
- type
- gravity
- cluster
- label
- position

Get Node Detail

GET /api/node/{id}

Returns full node metadata for the expansion panel.

Includes:

- description
- related nodes
- artifacts
- gravity explanation

Graph Query (Ask Interface)

POST /api/graph/query

Input:

```
{
  query: "What decisions shaped GetIT?"
}
```

Response:

```
{
  highlight_nodes: [],
  highlight_edges: [],
  explanation: ""
}
```

Used to highlight paths in the graph.

---

9. Performance Budgets

The engine must perform well on modern laptops.

Initial Load

Target:

< 2 seconds

Graph payload:

< 300 nodes

Rendering

Target:

60 FPS

Minimum acceptable:

45 FPS

Node Budget

Simultaneously rendered nodes:

≤ 300

Edge Budget

Simultaneously rendered edges:

≤ 800

Interaction Latency

Hover response:

< 50 ms

Click → panel open:

< 150 ms

Camera transition:

250–450 ms

---

10. Optimization Strategy

To maintain performance:

- use instanced meshes
- apply level-of-detail visibility rules
- hide low importance nodes at distance
- bundle weak edges
- disable animations when frame rate drops

Fallback order:

1. disable edge animation
2. reduce cluster particles

3. hide skill nodes
4. hide minor edges

---

11. Interaction Model

Supported interactions:

Hover node
Click node
Drag canvas
Zoom camera
Reset view

Hover Behavior

Hovering a node:

- increases glow
- highlights connected edges
- brightens connected nodes

Purpose:

Reveal graph structure.

Click Behavior

Clicking a node:

- centers camera
- opens Node Expansion Panel
- highlights local cluster

---

12. Visual Goals

The constellation must feel:

- calm
- spacious
- intelligent
- alive

Avoid:

- dashboard UI
- heavy labeling
- noisy graph layouts
- corporate aesthetics

The system should resemble a **knowledge universe**.

---

13. Acceptance Criteria

The engine is considered successful if the following are true.

Visual

- the graph feels beautiful and calm
- nodes glow subtly without visual clutter
- clusters are visually distinct

Cognitive

Within **10 seconds**, a visitor can identify:

- the central person node
- major project clusters

Within **2 minutes**, a visitor can understand:

- what this person builds
- which projects matter most

Interaction

- navigation feels smooth
- camera movement feels natural
- node exploration is intuitive

Product Impact

Visitors should naturally react with:

> “This is the most interesting profile I’ve seen.”

If users spend **3+ minutes exploring**, the system succeeds.

---

14. Strategic Role

This engine is the **atomic unit of the North Star network**.

If the single-profile constellation works, it becomes the template for:

- thousands
- millions
- billions

of capability nodes.

The system must therefore be designed with scalability in mind, but optimized for **one extraordinary first experience**.

End of Document

North Star Constellation Engine v0 — Single Profile

Constellation Engine Rendering Architecture

Constellation Engine Rendering Architecture

1) Data → Graph → Scene pipeline

1. **Fetch graph slice** (neighborhood or query result)
2. **Normalize nodes/edges** (types, weights, visibility)
3. **Compute layout** (2D/3D positions + cluster anchors)
4. **Build render buffers** (instanced meshes + line segments)
5. **Run simulation loop** (slow drift + hover/selection forces)
6. **Apply interaction state** (hover, select, spotlight)
7. **Render** (with progressive enhancement + FPS governor)

Layout + Clustering Math (the “map physics”)

A) Node positioning strategy (use zoom levels)

You do **not** run one force layout for everything. You run different strategies per zoom level.

Zoom Level 4: Node view (close)

- **Local force layout** only for a small neighborhood (50–300 nodes).
- Use **ForceAtlas2** or **D3-force** variant with constraints:
    - center gravity on selected node
    - collision radius by node type
    - edge weight as spring strength
    - cap iterations (fast)

Outcome: intimate, readable constellation around “Prentiss”.

Zoom Level 3: Constellation view (mid)

- Use **cluster-aware layout**:
    - compute communities (Louvain/Leiden) on the slice
    - position clusters with a coarse force layout
    - position nodes *within* cluster with local forces
- Add **edge bundling** for visual clarity (curved filaments instead of spaghetti)

Outcome: meaningful group shapes, fewer crossings.

Zoom Level 2: Cluster view (farther)

- Treat each cluster as a single “supernode”.
- Layout clusters on a stable plane:
    - force layout on supernodes
    - keep previous positions as priors (prevents jitter)
- Render only:
    - cluster hulls / glow clouds
    - top edges between clusters

Outcome: “galaxies” feel stable and navigable.

Zoom Level 1: Cosmos view (farthest)

- No force sim.
- Use a **fixed atlas layout**:
    - place major domains (AI, marketplaces, edtech…) in predetermined regions
    - clusters settle into these regions over time
- Render as “nebula fields” with sparse points.

Outcome: cosmic map with recognizable regions.

B) Clustering (community detection)

Use **Leiden** (preferred) or **Louvain** on the graph slice.

- Input: nodes + weighted edges
- Output: community id per node
- Weight edges based on:
    - shared decisions (high)
    - shared patterns (high)
    - project similarity (medium)
    - co-citation / co-artifact (medium)
    - person-person collaboration (later)

Store cluster ids as a derived field per node per “graph snapshot”.

C) Edge weighting + edge types (visual meaning)

Edge types become different filament behaviors:

- `contains` (project → claim): thin, short-range, dim
- `supports` (claim → artifact): short, dotted / dashed

- `similar_to` (claim ↔ claim): long-range, brighter, bundled
- `pattern_member` (node → pattern): faint, only visible in pattern mode
- `built` (person → project): primary strong filament

Edge weight controls:

- brightness
- thickness
- curvature intensity
- bundling priority

D) Edge bundling (to prevent spaghetti)

Use **Force-Directed Edge Bundling** on the current slice.

- Compute control points for each edge
- Render as quadratic/cubic bezier curves
- Bundle edges that share endpoints or traverse similar routes

This gives you that “flowing filament” look from your first reference image.

Rendering Strategy (WebGL that doesn’t melt laptops)

Core constraints

- Must handle thousands of nodes smoothly
- Must degrade gracefully on weaker devices
- Must respect Reduce Motion

Approach

- **React Three Fiber** for scene
- Use **instancing** for nodes:
    - one mesh per node type
    - 1 draw call per type
- Use **LineSegments** or GPU polyline library for edges
- Use **LOD** (level of detail):
    - far zoom: render clusters only
    - mid zoom: render clusters + top-k edges
    - close zoom: render full neighborhood

FPS governor

- Target 60fps if possible
- If FPS < 45 for 1s:
    - reduce particles/glow
    - reduce edge count
    - disable expensive postprocessing
- If FPS < 30:
    - switch to “simple mode” (static points + minimal lines)

Postprocessing (only when device can handle)

- subtle bloom (very restrained)
- vignette
- slight chromatic aberration (optional, minimal)
- noise grain overlay (CSS layer, cheap)

Motion Design Implementation (meaningful physics)

Motion should be **slow** and **semantic**.

Base drift

- Perlin noise offsets on clusters
- nodes within cluster drift slightly around anchor

Selection spotlight

When a node is selected:

- apply gravitational pull toward center
- brighten incident edges
- fade non-neighborhood nodes

Hover halo

- increase emissive intensity
- slight scale bump (1.03 max)
- show preview card

State Model (so interactions are clean)

You need a “graph UI state” store:

- `mode`: people/projects/decisions/patterns
- `zoomLevel`: 1–4
- `selectedNodeId`
- `hoverNodeId`
- `filters`: domain, status, verified-only
- `queryActive`: ask-the-graph result active?
- `spotlightSet`: node ids to emphasize

What I need to hand to your CTO (simple list)

A) Product surfaces (pages)

- Profile hub page (Prentiss)
- Project pages (Build Ledger)
- Ask page (RAG chat)
- Fit Check page
- Constellation canvas page (the map)
- “Coming soon” placeholder panels for future nodes

B) Data + backend

- Postgres/Supabase schema migrations:
  - person
  - project
  - claim
  - artifact
  - citation
  - embedding
  - edge (Phase 2+)
  - pattern + pattern_membership (Phase 3+)
- Storage setup for artifacts (Supabase Storage)
- Ingestion pipeline:

    - MDX ingestion
    - upload ingestion
    - claim extraction job (LLM)
    - citation anchoring
- Embedding pipeline + vector index
- Retrieval API:
    - `/api/ask` (RAG with citations)
    - `/api/fit-check` (structured eval)
    - `/api/graph/query` (returns a subgraph)
    - `/api/graph/neighborhood/:id` (node neighborhood)
- Background jobs/queue (for embeddings, extraction, later edges/patterns)

C) Frontend + WebGL

- React Three Fiber scene
- Node rendering system (instanced meshes)
- Edge rendering system (bundled curved filaments)
- Zoom system (LOD rules per level)
- Interaction layer (hover/select/pan/zoom)
- Right-side panel system (node details)
- Search/command bar overlay
- “Ask the graph” overlay + highlight logic
- Performance governor + reduce-motion support

D) Algorithms / math layer

- Local force layout for neighborhoods
- Community detection (Leiden/Louvain) for slices
- Edge bundling for visual clarity
- Stable positioning (layout priors to prevent jitter)
- Node sizing and edge weight mapping rules

E) Design system (so it stays calm)

- Node type visual specs (shape, color, glow)
- Typography system for panels (minimal)
- Motion rules (timings, easing)
- Accessibility rules (reduce motion, contrast)

F) Testing / acceptance

- FPS and memory budgets
- Snapshot tests for graph API outputs
- RAG answer citation enforcement
- Security tests (RLS for private artifacts)

---

skill needs

Find skills.md to handle these tasks then install them.

Task Type / Skill	Best Tool(s) to Use
Web research, fact-checking, browsing articles or documents online	`browser.search` to find up-to-date information; `browser.open` and `browser.find` to read and extract content; if images are needed, `image_gen` can generate high-level graphics but is not for real-world images.
Working with user-uploaded files (docs, PDFs) or internal resources	`file_search.msearch` to locate documents; `file_search.mclick` to open them. When the information is on the user's computer (locally), use the `computer` tool to navigate and open files.
Scripting, data analysis, and code execution	`python_user_visible` for running Python code that produces output visible to the user (e.g. charts, tables); `container` for more technical shell commands and file manipulation.
Using or generating structured documents (reports, slides)	`container` and `python_user_visible` to run scripts that create PowerPoint, PDF, or Markdown reports; `canmore` for collaborative document editing in a canvas.
Image creation for illustrative purposes	`image_gen` for generating abstract or decorative images; it's not suited for realistic photos—real-world images must be fetched via web search (and cited).
Automated reminders or recurring tasks	`automations` for scheduling reminders and recurring checks (e.g. daily summaries or weekly searches).
Working with personal context or saving user preferences	`personal_context` to retrieve prior user-specific details; `bio.update` to save or forget information about the user for future conversations.

Constellation Graph Engine

North Star — Constellation Graph Engine

Rendering Architecture Spec

Audience: CTO

1) Goal

Build a graph engine that makes a **single capability constellation** feel extraordinary on day one, while using an architecture that can expand into a dense multi-profile universe later.

The engine must do five things well:

- render smoothly on mainstream laptops
- feel calm, premium, and spatial
- make relationships legible in under 10 seconds
- support progressive disclosure through zoom
- scale from one profile to many without re-architecture

The recommended stack is:

- **Next.js + React**
- **three.js**
- **@react-three/fiber**
- **@react-three/drei**
- **graphology**
- **graphology-layout-forceatlas2**
- **d3-force** for local neighborhood re-layout
- **zustand** or equivalent for graph UI state

Why this stack:

- React Three Fiber is the React renderer for three.js and is designed to let scenes participate cleanly in React state and composition. (Poimandres Documentation)
- three.js provides the core rendering primitives you need here, including `InstancedMesh`, `LOD`, points, lines, and custom shaders through `ShaderMaterial`. (Three.js)
- graphology is a robust JavaScript/TypeScript graph object with layout and metrics packages, which is a better long-term graph backbone than inventing one ad hoc. (Graphology)

- ForceAtlas2 in graphology is a strong fit for stable graph layouts, with explicit support in the standard library. (Graphology)
- d3-force gives a flexible force simulation based on a velocity Verlet integrator, which is useful for small local re-layouts around the selected node. (D3.js)

---

2) Exact Library Decisions

Rendering

Use:

- `three`
- `@react-three/fiber`
- `@react-three/drei`

Do not use a generic graph visualization library as the main renderer. The product needs a cinematic, domain-specific visual language, not an off-the-shelf network chart.

Graph model + layout

Use:

- `graphology`
- `graphology-layout-forceatlas2`
- `graphology-layout`
- optionally `graphology-metrics` later
- `d3-force` for local interaction-driven adjustments only

State

Use:

- `zustand` for UI state
- React Query or native fetch caching for graph payloads

Why not Sigma as primary renderer

Sigma is good for graph exploration, but North Star needs full control over:

- depth
- shaders

- glow
- nonstandard node geometry
- cinematic motion
- gradual universe reveal

For this product, Sigma is too constrained.

---

3) Scene Model

The renderer is split into **five visual layers**.

Layer A — Background Field

Purpose:

- subtle cosmic depth
- mood
- no semantic content

Implementation:

- fullscreen gradient plane
- optional low-cost starfield points layer
- optional noise overlay in CSS

Layer B — Cluster Atmosphere

Purpose:

- indicate domain grouping
- create nebula-like cluster identity

Implementation:

- translucent billboard sprites or soft point clouds
- one field per cluster
- opacity tied to cluster aggregate gravity

Layer C — Edges

Purpose:

- show relationships without spaghetti

Implementation:

- bezier or Catmull-Rom curve paths
- particle flow or animated alpha pulse on selected edges
- only top-weighted edges visible at far zoom

D3's curve utilities are useful conceptually for smooth interpolation if you generate 2D control paths before sending them to WebGL. (D3.js)

Layer D — Nodes

Purpose:

- semantic graph units

Implementation:

- instanced geometry per node type
- separate draw group per node family

Layer E — Interaction / HUD

Purpose:

- hover, selection, labels, node cards, ask overlay

Implementation:

- React DOM overlay above the canvas
- no text rendering in 3D except possibly minimal labels at low volume

---

4) Node Types + Render Rules

Person node

Visual:

- brightest
- spherical core with soft halo
- warm white to gold tint

Geometry:

- instanced sphere

Project node

Visual:

- medium prominence
- cyan / electric blue family
- slightly larger than decisions

Geometry:

- instanced icosahedron or sphere

Decision node

Visual:

- magenta / violet
- sharper silhouette

Geometry:

- instanced octahedron or rotated box

Skill node

Visual:

- small teal points
- understated

Geometry:

- instanced small sphere or point sprite

Outcome node

Visual:

- amber / orange
- higher contrast than skills

Geometry:

- instanced box or rounded square billboard

Ghost / future nodes

Visual:

- faint, translucent
- low alpha
- minimal hover response

Purpose:

- hint at the much larger universe without pretending it exists yet

---

5) Node Data Schema

Use a normalized runtime schema. Keep it simple.

```
type NodeType =
  | 'person'
  | 'project'
  | 'decision'
  | 'skill'
  | 'outcome'
  | 'ghost';

type GraphNode = {
  id: string;
  type: NodeType;
  label: string;
  shortDescription?: string;

  // semantic grouping
  clusterId?: string;
  domain?: string;

  // graph and render state
  gravity: number;          // normalized 0..1
  size: number;             // computed display scalar
  importance: number;       // ranking for label visibility
  state?: 'dormant' | 'active' | 'significant' | 'anchor';

  // layout
  position: [number, number, number];
  velocity?: [number, number, number];
  fixed?: boolean;
```

```
  // visual customization
  color?: string;
  halo?: number;
  opacity?: number;

  // content relationships
  connectedNodeIds?: string[];

  // drill-in
  panelDataRef?: string;

  // analytics
  createdAt?: string;
  updatedAt?: string;
};
```

Edge schema

```
type EdgeType =
  | 'built'
  | 'contains'
  | 'supports'
  | 'influences'
  | 'results_in'
  | 'related_to'
  | 'ghost_link';

type GraphEdge = {
  id: string;
  source: string;
  target: string;
  type: EdgeType;
  weight: number;            // normalized 0..1
  clusterBridge?: boolean;   // true if connecting clusters
  animated?: boolean;
  opacity?: number;
};
```

Cluster schema

```
type GraphCluster = {
  id: string;
```

```
  label: string;
  colorA: string;
  colorB: string;
  centroid: [number, number, number];
  radius: number;
  gravity: number;
  nodeIds: string[];
};
```

---

6) GPU Pipeline

CPU responsibilities

- fetch graph slice
- compute or load layout positions
- assign cluster membership
- compute node size/visibility tiers
- compute edge visibility tiers
- create packed typed arrays for GPU upload

GPU responsibilities

- draw instanced nodes
- draw bundled edge curves
- animate halos / pulses
- render cluster atmospheres
- support selection and hover highlighting

Draw strategy

Use **instancing** aggressively.

- one `InstancedMesh` for person nodes
- one for projects
- one for decisions
- one for outcomes
- one point cloud or instanced mesh for skills

This minimizes draw calls and is directly supported by three.js through `InstancedMesh`. (Three.js)

LOD strategy

Use three.js LOD concepts and your own visibility thresholds.

At far zoom:

- hide low-importance skills
- collapse many edges
- render cluster atmospheres + anchor nodes only

At medium zoom:

- show project and decision structure
- hide most weak edges

At close zoom:

- render local neighborhood in full

three.js includes `LOD` as a first-class primitive, which is useful for zoom-sensitive object simplification. (Three.js)

---

7) Layout Architecture

Do not run one monolithic force simulation for every state. Use a hybrid.

Global layout

For the one-profile MFP:

- precompute a stable base layout for the founder constellation
- store positions with the graph payload
- keep person node anchored at origin or a deliberate central location

Local neighborhood layout

When a node is selected:

- run a limited local d3-force pass on only the nearby subgraph
- re-space child nodes for readability
- keep anchor nodes pinned

d3-force is appropriate for this because it is modular and lightweight for small local simulations. (D3.js)

Later multi-profile layout

For cluster-level layout:

- use graphology + ForceAtlas2
- initialize positions first; ForceAtlas2 requires x and y starting coordinates and does not behave correctly if everything starts at zero. (Graphology)

Cluster placement

Use a two-step model:

1. compute communities
2. place communities as supernodes
3. place members relative to cluster centroid

This is critical. It prevents the graph from becoming a random cloud.

---

8) Clustering Algorithms

v0 / single profile

Use rule-based clustering:

- one cluster per project
- optional subclusters by node type

This is enough for the first build.

v1 / multi-profile

Use:

- community detection over weighted edges
- cluster score derived from:
    - project overlap
    - decision similarity
    - outcome similarity
    - shared domain tags

Suggested algorithm path

- start with simple weighted connected-component or label propagation
- move to Louvain/Leiden later
- keep ForceAtlas2 for placement, not for semantic clustering alone

Edge bundling

For visual clarity:

- generate edge bundles for project-centered relationships
- weak edges should merge into filaments
- strong edges can render directly

This matters more than exotic physics. Good bundling is the difference between elegance and noise.

---

9) Shader Strategy

Use custom shaders only where they materially improve the feel.

Materials

- standard three.js materials for fast iteration
- custom `ShaderMaterial` for:
    - node glow
    - animated halo
    - edge pulse
    - cluster atmosphere
- use `RawShaderMaterial` only if you need total control

three.js documents that `ShaderMaterial` works with `WebGLRenderer`, and `RawShaderMaterial` is the lower-level option if you do not want built-in uniforms/attributes injected. (Three.js)

Example: node glow fragment shader

```
// fragment.glsl
precision mediump float;

uniform vec3 uColor;
uniform float uOpacity;
```

```
varying vec2 vUv;

void main() {
  vec2 p = vUv - 0.5;
  float d = length(p) * 2.0;

  // Bright core + soft falloff
  float core = smoothstep(0.35, 0.0, d);
  float halo = smoothstep(1.0, 0.2, d) * 0.35;

  float alpha = max(core, halo) * uOpacity;
  vec3 color = uColor * (core * 1.3 + halo);

  gl_FragColor = vec4(color, alpha);
}
```

Example: node glow vertex shader

```
// vertex.glsl
precision mediump float;

attribute vec3 position;
attribute vec2 uv;

uniform mat4 modelViewMatrix;
uniform mat4 projectionMatrix;
uniform float uScale;

varying vec2 vUv;

void main() {
  vUv = uv;
  vec3 scaled = position * uScale;
  gl_Position = projectionMatrix * modelViewMatrix * vec4(scaled, 1.0);
}
```

Example: edge pulse fragment shader

```
precision mediump float;

uniform vec3 uColor;
uniform float uTime;
uniform float uActive;
```

```
varying float vLineU;

void main() {
  float pulse = sin((vLineU * 12.0) - (uTime * 2.5)) * 0.5 + 0.5;
  float intensity = mix(0.25, 1.0, pulse) * uActive;
  gl_FragColor = vec4(uColor * intensity, intensity);
}
```

Use cases

- node cores should feel soft, not neon
- pulses should be subtle and event-driven
- background should remain dark and calm

---

10) Interaction Engine

Input system

Support:

- hover
- click
- drag/pan
- scroll zoom
- double-click focus
- reset view

Selection behavior

On click:

- set selected node
- center camera smoothly
- brighten node and first-degree neighbors
- dim unrelated nodes
- open node expansion panel

Hover behavior

On hover:

- raise emissive / halo slightly
- preview card in DOM overlay
- temporarily brighten incident edges

Zoom behavior

Use discrete semantic zoom bands:

- band 0: universe hint
- band 1: profile constellation
- band 2: project cluster
- band 3: node deep dive

At each band:

- rerun visibility rules
- do not just scale everything uniformly

State model

Use a dedicated graph UI store:

```
type GraphUIState = {
  selectedNodeId: string | null;
  hoveredNodeId: string | null;
  zoomBand: 0 | 1 | 2 | 3;
  filters: {
    nodeTypes: NodeType[];
    showGhosts: boolean;
  };
  askQuery: string | null;
  spotlightNodeIds: string[];
};
```

DOM overlay

The canvas should not own all UI.
Keep:

- search / ask bar
- node expansion panel
- tooltips
- legend

- share button
  in React DOM overlays above the canvas.

That gives you cleaner typography, accessibility, and lower GPU overhead.

---

11) Ask-the-Graph Rendering Behavior

Ask must not feel like “chat bolted onto a graph.”

Input

Persistent command bar above canvas:

- “Ask the graph…”

On submit

The engine should:

- resolve relevant nodes and edges
- spotlight them
- animate path reveal
- open explanation card

Example

Query:

- “What decisions shaped GetIT the most?”

Expected visual response:

- project node spotlighted
- top decision nodes brighten
- decision→outcome edges pulse once
- answer panel opens with top 3 reasons

This is where the graph becomes a reasoning surface instead of a decorative object.

---

12) Performance Budgets

For the one-profile MFP, lock these budgets:

First meaningful render

- target: **< 2.0s**

Stable frame rate

- target: **55–60 FPS** on modern laptop
- floor: **45 FPS**

Scene budgets

- nodes visible at once: **<= 300**
- edges visible at once: **<= 800**
- total graph payload: **<= 1,500 structural items** for founder node including ghost nodes

Interaction latency

- hover response: **< 50ms**
- click to panel open: **< 150ms**
- camera focus transition: **250–450ms**

Fallback behavior

If FPS drops:

1. disable animated edge particles
2. reduce cluster atmosphere opacity
3. hide low-priority node labels
4. thin weak edges

---

13) API Contracts Needed

`GET /api/graph/profile/:id`

Returns the full founder graph slice.


```
{
  "profile": {
    "id": "prentiss",
    "label": "Prentiss",
```

```
    "headline": "Frontier Operator"
  },
  "nodes": [],
  "edges": [],
  "clusters": [],
  "camera": {
    "position": [0, 0, 18],
    "target": [0, 0, 0]
  }
}
```


`GET /api/graph/node/:id`

Returns node detail for the expansion panel.

`POST /api/graph/query`

Accepts ask-the-graph prompt and returns:

- spotlight node ids
- spotlight edge ids
- textual explanation
- optional cluster emphasis

`POST /api/graph/layout/recompute`

Admin-only in v0.
Used to re-run founder layout after content changes.

---

14) Build Sequence

Phase 1 — Foundation

- set up R3F scene
- render static founder node + 3 project nodes
- implement dark field + node materials

Phase 2 — Graph structure

- add node schema and edges

- render project mini-constellations
- add hover and click

Phase 3 — Panel + ask

- node expansion panel
- ask-the-graph query highlighting
- camera transitions

Phase 4 — polish

- ghost nodes
- cluster atmosphere
- edge bundling
- performance governor
- reduce motion support

---

15) Acceptance Criteria

Visual

- the first load feels premium, calm, and spatial
- one person's graph feels rich enough to explore for several minutes
- the universe hint is visible without feeling fake

Cognitive

- user can identify the center node immediately
- user can understand project cluster structure immediately
- user can see what matters most without instruction

Technical

- frame rate stays within budget
- interactions feel smooth
- no layout jitter on idle
- no text illegibility due to over-rendering

Product

- users say "I want one"
- graph clearly implies a much larger system

- the founder node stands on its own even before multi-profile scale

---

16) Final Recommendation

Build this as a **deliberately constrained cinematic graph engine**, not a general-purpose graph widget.

That means:

- fewer node types
- better semantic hierarchy
- stronger render discipline
- careful animation
- one extraordinary founder constellation

If the one-node experience feels profound, the expansion path is obvious.

If it feels like a clever graph demo, the product is dead.

If you want, I'll turn this next into the **engineering handoff pack**:

- package.json dependencies
- folder architecture
- TypeScript interfaces
- endpoint contracts
- starter R3F scene skeleton

Rendering Architecture Spec

North Star — Constellation Graph Engine

Rendering Architecture Spec

Audience: CTO

1) Goal

Build a graph engine that makes a **single capability constellation** feel extraordinary on day one, while using an architecture that can expand into a dense multi-profile universe later.

The engine must do five things well:

- render smoothly on mainstream laptops
- feel calm, premium, and spatial
- make relationships legible in under 10 seconds
- support progressive disclosure through zoom
- scale from one profile to many without re-architecture

The recommended stack is:

- **Next.js + React**
- **three.js**
- **@react-three/fiber**
- **@react-three/drei**
- **graphology**
- **graphology-layout-forceatlas2**
- **d3-force** for local neighborhood re-layout
- **zustand** or equivalent for graph UI state

Why this stack:

- React Three Fiber is the React renderer for three.js and is designed to let scenes participate cleanly in React state and composition. (Poimandres Documentation)
- three.js provides the core rendering primitives you need here, including `InstancedMesh`, `LOD`, points, lines, and custom shaders through `ShaderMaterial`. (Three.js)
- graphology is a robust JavaScript/TypeScript graph object with layout and metrics packages, which is a better long-term graph backbone than inventing one ad hoc. (Graphology)

- ForceAtlas2 in graphology is a strong fit for stable graph layouts, with explicit support in the standard library. (Graphology)
- d3-force gives a flexible force simulation based on a velocity Verlet integrator, which is useful for small local re-layouts around the selected node. (D3.js)

---

2) Exact Library Decisions

Rendering

Use:

- `three`
- `@react-three/fiber`
- `@react-three/drei`

Do not use a generic graph visualization library as the main renderer. The product needs a cinematic, domain-specific visual language, not an off-the-shelf network chart.

Graph model + layout

Use:

- `graphology`
- `graphology-layout-forceatlas2`
- `graphology-layout`
- optionally `graphology-metrics` later
- `d3-force` for local interaction-driven adjustments only

State

Use:

- `zustand` for UI state
- React Query or native fetch caching for graph payloads

Why not Sigma as primary renderer

Sigma is good for graph exploration, but North Star needs full control over:

- depth
- shaders

- glow
- nonstandard node geometry
- cinematic motion
- gradual universe reveal

For this product, Sigma is too constrained.

---

3) Scene Model

The renderer is split into **five visual layers**.

Layer A — Background Field

Purpose:

- subtle cosmic depth
- mood
- no semantic content

Implementation:

- fullscreen gradient plane
- optional low-cost starfield points layer
- optional noise overlay in CSS

Layer B — Cluster Atmosphere

Purpose:

- indicate domain grouping
- create nebula-like cluster identity

Implementation:

- translucent billboard sprites or soft point clouds
- one field per cluster
- opacity tied to cluster aggregate gravity

Layer C — Edges

Purpose:

- show relationships without spaghetti

Implementation:

- bezier or Catmull-Rom curve paths
- particle flow or animated alpha pulse on selected edges
- only top-weighted edges visible at far zoom

D3's curve utilities are useful conceptually for smooth interpolation if you generate 2D control paths before sending them to WebGL. (D3.js)

Layer D — Nodes

Purpose:

- semantic graph units

Implementation:

- instanced geometry per node type
- separate draw group per node family

Layer E — Interaction / HUD

Purpose:

- hover, selection, labels, node cards, ask overlay

Implementation:

- React DOM overlay above the canvas
- no text rendering in 3D except possibly minimal labels at low volume

---

4) Node Types + Render Rules

Person node

Visual:

- brightest
- spherical core with soft halo
- warm white to gold tint

Geometry:

- instanced sphere

Project node

Visual:

- medium prominence
- cyan / electric blue family
- slightly larger than decisions

Geometry:

- instanced icosahedron or sphere

Decision node

Visual:

- magenta / violet
- sharper silhouette

Geometry:

- instanced octahedron or rotated box

Skill node

Visual:

- small teal points
- understated

Geometry:

- instanced small sphere or point sprite

Outcome node

Visual:

- amber / orange
- higher contrast than skills

Geometry:

- instanced box or rounded square billboard

Ghost / future nodes

Visual:

- faint, translucent
- low alpha
- minimal hover response

Purpose:

- hint at the much larger universe without pretending it exists yet

---

5) Node Data Schema

Use a normalized runtime schema. Keep it simple.

```
type NodeType =
  | 'person'
  | 'project'
  | 'decision'
  | 'skill'
  | 'outcome'
  | 'ghost';

type GraphNode = {
  id: string;
  type: NodeType;
  label: string;
  shortDescription?: string;

  // semantic grouping
  clusterId?: string;
  domain?: string;

  // graph and render state
  gravity: number;          // normalized 0..1
  size: number;             // computed display scalar
  importance: number;       // ranking for label visibility
  state?: 'dormant' | 'active' | 'significant' | 'anchor';

  // layout
  position: [number, number, number];
  velocity?: [number, number, number];
  fixed?: boolean;
```

```
  // visual customization
  color?: string;
  halo?: number;
  opacity?: number;

  // content relationships
  connectedNodeIds?: string[];

  // drill-in
  panelDataRef?: string;

  // analytics
  createdAt?: string;
  updatedAt?: string;
};
```

Edge schema

```
type EdgeType =
  | 'built'
  | 'contains'
  | 'supports'
  | 'influences'
  | 'results_in'
  | 'related_to'
  | 'ghost_link';

type GraphEdge = {
  id: string;
  source: string;
  target: string;
  type: EdgeType;
  weight: number;            // normalized 0..1
  clusterBridge?: boolean;   // true if connecting clusters
  animated?: boolean;
  opacity?: number;
};
```

Cluster schema

```
type GraphCluster = {
  id: string;
```

```
  label: string;
  colorA: string;
  colorB: string;
  centroid: [number, number, number];
  radius: number;
  gravity: number;
  nodeIds: string[];
};
```

---

6) GPU Pipeline

CPU responsibilities

- fetch graph slice
- compute or load layout positions
- assign cluster membership
- compute node size/visibility tiers
- compute edge visibility tiers
- create packed typed arrays for GPU upload

GPU responsibilities

- draw instanced nodes
- draw bundled edge curves
- animate halos / pulses
- render cluster atmospheres
- support selection and hover highlighting

Draw strategy

Use **instancing** aggressively.

- one `InstancedMesh` for person nodes
- one for projects
- one for decisions
- one for outcomes
- one point cloud or instanced mesh for skills

This minimizes draw calls and is directly supported by three.js through `InstancedMesh`. (Three.js)

LOD strategy

Use three.js LOD concepts and your own visibility thresholds.

At far zoom:

- hide low-importance skills
- collapse many edges
- render cluster atmospheres + anchor nodes only

At medium zoom:

- show project and decision structure
- hide most weak edges

At close zoom:

- render local neighborhood in full

three.js includes `LOD` as a first-class primitive, which is useful for zoom-sensitive object simplification. (Three.js)

---

7) Layout Architecture

Do not run one monolithic force simulation for every state. Use a hybrid.

Global layout

For the one-profile MFP:

- precompute a stable base layout for the founder constellation
- store positions with the graph payload
- keep person node anchored at origin or a deliberate central location

Local neighborhood layout

When a node is selected:

- run a limited local d3-force pass on only the nearby subgraph
- re-space child nodes for readability
- keep anchor nodes pinned

d3-force is appropriate for this because it is modular and lightweight for small local simulations. (D3.js)

Later multi-profile layout

For cluster-level layout:

- use graphology + ForceAtlas2
- initialize positions first; ForceAtlas2 requires x and y starting coordinates and does not behave correctly if everything starts at zero. (Graphology)

Cluster placement

Use a two-step model:

1. compute communities
2. place communities as supernodes
3. place members relative to cluster centroid

This is critical. It prevents the graph from becoming a random cloud.

---

8) Clustering Algorithms

v0 / single profile

Use rule-based clustering:

- one cluster per project
- optional subclusters by node type

This is enough for the first build.

v1 / multi-profile

Use:

- community detection over weighted edges
- cluster score derived from:
    - project overlap
    - decision similarity
    - outcome similarity
    - shared domain tags

Suggested algorithm path

- start with simple weighted connected-component or label propagation
- move to Louvain/Leiden later
- keep ForceAtlas2 for placement, not for semantic clustering alone

Edge bundling

For visual clarity:

- generate edge bundles for project-centered relationships
- weak edges should merge into filaments
- strong edges can render directly

This matters more than exotic physics. Good bundling is the difference between elegance and noise.

---

9) Shader Strategy

Use custom shaders only where they materially improve the feel.

Materials

- standard three.js materials for fast iteration
- custom `ShaderMaterial` for:
    - node glow
    - animated halo
    - edge pulse
    - cluster atmosphere
- use `RawShaderMaterial` only if you need total control

three.js documents that `ShaderMaterial` works with `WebGLRenderer`, and `RawShaderMaterial` is the lower-level option if you do not want built-in uniforms/attributes injected. (Three.js)

Example: node glow fragment shader

```
// fragment.glsl
precision mediump float;

uniform vec3 uColor;
uniform float uOpacity;
```

```
varying vec2 vUv;

void main() {
  vec2 p = vUv - 0.5;
  float d = length(p) * 2.0;

  // Bright core + soft falloff
  float core = smoothstep(0.35, 0.0, d);
  float halo = smoothstep(1.0, 0.2, d) * 0.35;

  float alpha = max(core, halo) * uOpacity;
  vec3 color = uColor * (core * 1.3 + halo);

  gl_FragColor = vec4(color, alpha);
}
```

Example: node glow vertex shader

```
// vertex.glsl
precision mediump float;

attribute vec3 position;
attribute vec2 uv;

uniform mat4 modelViewMatrix;
uniform mat4 projectionMatrix;
uniform float uScale;

varying vec2 vUv;

void main() {
  vUv = uv;
  vec3 scaled = position * uScale;
  gl_Position = projectionMatrix * modelViewMatrix * vec4(scaled, 1.0);
}
```

Example: edge pulse fragment shader

```
precision mediump float;

uniform vec3 uColor;
uniform float uTime;
uniform float uActive;
```

```
varying float vLineU;

void main() {
  float pulse = sin((vLineU * 12.0) - (uTime * 2.5)) * 0.5 + 0.5;
  float intensity = mix(0.25, 1.0, pulse) * uActive;
  gl_FragColor = vec4(uColor * intensity, intensity);
}
```

Use cases

- node cores should feel soft, not neon
- pulses should be subtle and event-driven
- background should remain dark and calm

---

10) Interaction Engine

Input system

Support:

- hover
- click
- drag/pan
- scroll zoom
- double-click focus
- reset view

Selection behavior

On click:

- set selected node
- center camera smoothly
- brighten node and first-degree neighbors
- dim unrelated nodes
- open node expansion panel

Hover behavior

On hover:

- raise emissive / halo slightly
- preview card in DOM overlay
- temporarily brighten incident edges

Zoom behavior

Use discrete semantic zoom bands:

- band 0: universe hint
- band 1: profile constellation
- band 2: project cluster
- band 3: node deep dive

At each band:

- rerun visibility rules
- do not just scale everything uniformly

State model

Use a dedicated graph UI store:

```
type GraphUIState = {
  selectedNodeId: string | null;
  hoveredNodeId: string | null;
  zoomBand: 0 | 1 | 2 | 3;
  filters: {
    nodeTypes: NodeType[];
    showGhosts: boolean;
  };
  askQuery: string | null;
  spotlightNodeIds: string[];
};
```

DOM overlay

The canvas should not own all UI.
Keep:

- search / ask bar
- node expansion panel
- tooltips
- legend

- share button
  in React DOM overlays above the canvas.

That gives you cleaner typography, accessibility, and lower GPU overhead.

---

11) Ask-the-Graph Rendering Behavior

Ask must not feel like “chat bolted onto a graph.”

Input

Persistent command bar above canvas:

- “Ask the graph…”

On submit

The engine should:

- resolve relevant nodes and edges
- spotlight them
- animate path reveal
- open explanation card

Example

Query:

- “What decisions shaped GetIT the most?”

Expected visual response:

- project node spotlighted
- top decision nodes brighten
- decision→outcome edges pulse once
- answer panel opens with top 3 reasons

This is where the graph becomes a reasoning surface instead of a decorative object.

---

12) Performance Budgets

For the one-profile MFP, lock these budgets:

First meaningful render

- target: **< 2.0s**

Stable frame rate

- target: **55–60 FPS** on modern laptop
- floor: **45 FPS**

Scene budgets

- nodes visible at once: **<= 300**
- edges visible at once: **<= 800**
- total graph payload: **<= 1,500 structural items** for founder node including ghost nodes

Interaction latency

- hover response: **< 50ms**
- click to panel open: **< 150ms**
- camera focus transition: **250–450ms**

Fallback behavior

If FPS drops:

1. disable animated edge particles
2. reduce cluster atmosphere opacity
3. hide low-priority node labels
4. thin weak edges

---

13) API Contracts Needed

`GET /api/graph/profile/:id`

Returns the full founder graph slice.


```
{
  "profile": {
    "id": "prentiss",
    "label": "Prentiss",
```

```
    "headline": "Frontier Operator"
  },
  "nodes": [],
  "edges": [],
  "clusters": [],
  "camera": {
    "position": [0, 0, 18],
    "target": [0, 0, 0]
  }
}
```


`GET /api/graph/node/:id`

Returns node detail for the expansion panel.

`POST /api/graph/query`

Accepts ask-the-graph prompt and returns:

- spotlight node ids
- spotlight edge ids
- textual explanation
- optional cluster emphasis

`POST /api/graph/layout/recompute`

Admin-only in v0.
Used to re-run founder layout after content changes.

---

14) Build Sequence

Phase 1 — Foundation

- set up R3F scene
- render static founder node + 3 project nodes
- implement dark field + node materials

Phase 2 — Graph structure

- add node schema and edges

- render project mini-constellations
- add hover and click

Phase 3 — Panel + ask

- node expansion panel
- ask-the-graph query highlighting
- camera transitions

Phase 4 — polish

- ghost nodes
- cluster atmosphere
- edge bundling
- performance governor
- reduce motion support

---

15) Acceptance Criteria

Visual

- the first load feels premium, calm, and spatial
- one person's graph feels rich enough to explore for several minutes
- the universe hint is visible without feeling fake

Cognitive

- user can identify the center node immediately
- user can understand project cluster structure immediately
- user can see what matters most without instruction

Technical

- frame rate stays within budget
- interactions feel smooth
- no layout jitter on idle
- no text illegibility due to over-rendering

Product

- users say "I want one"
- graph clearly implies a much larger system

- the founder node stands on its own even before multi-profile scale

---

16) Final Recommendation

Build this as a **deliberately constrained cinematic graph engine**, not a general-purpose graph widget.

That means:

- fewer node types
- better semantic hierarchy
- stronger render discipline
- careful animation
- one extraordinary founder constellation

If the one-node experience feels profound, the expansion path is obvious.

If it feels like a clever graph demo, the product is dead.

turn this next into the **engineering handoff pack**:

- package.json dependencies
- folder architecture
- TypeScript interfaces
- endpoint contracts
- starter R3F scene skeleton